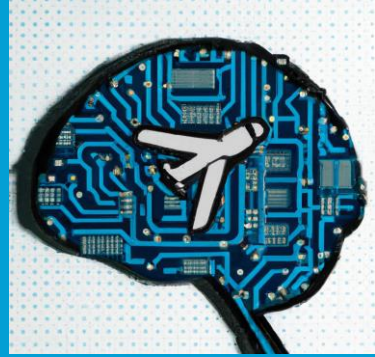
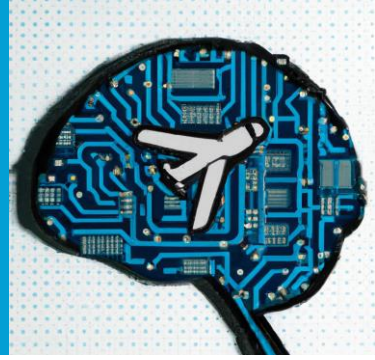




Generative Artificial Intelligence

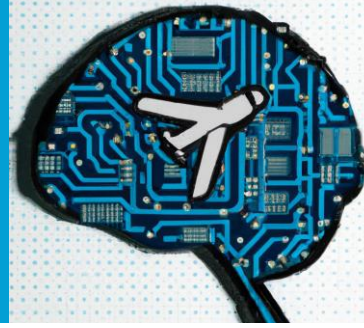
Lecture objectives



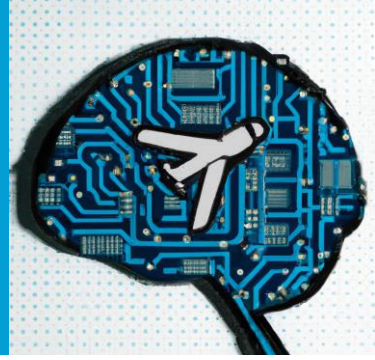
- Motivate the study of generative AI
- Students can list and describe different approaches to generative AI with their advantages and disadvantages
- Students can apply generative AI methods



Which face has been generated with AI?



Which face has been generated with AI?

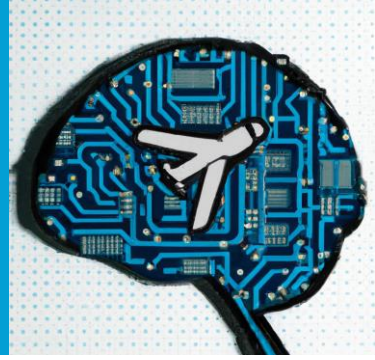


Answer: all of them!

Faces from [AI-Generated Faces Images](#)

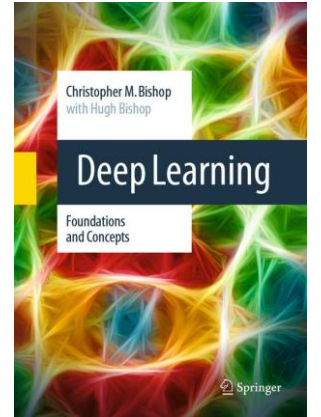
<https://thispersondoesnotexist.com/>

Recommended reading



“Deep learning: Foundations and concepts”, by Bishop, C. M., & Bishop, H. (2023). Available via TU Delft [here](#).

Chapter 17, Section 19.2, Chapter 20.

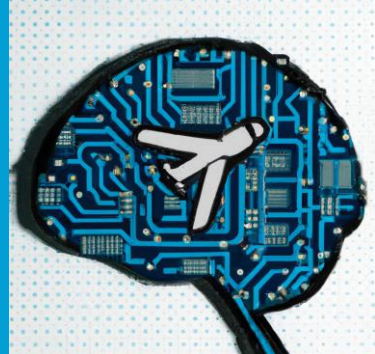


Interactive reader on github:

https://github.com/guidoAI/generative_AI

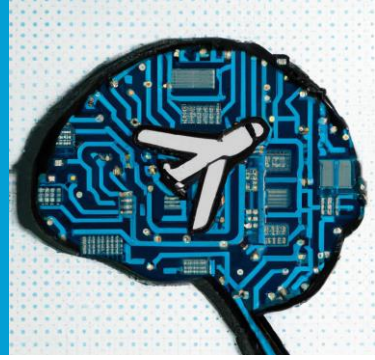
https://mybinder.org/v2/gh/guidoAI/generative_AI/HEAD?urlpath=%2Fdoc%2Ftree%2FGenerative+AI.ipynb





Generative models

Discriminatory vs generative models



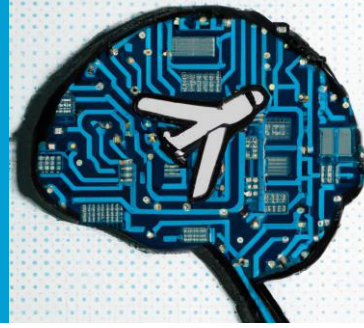
Discriminatory

Models that learn to map input samples to target outputs, e.g., for classification or regression. $P(\mathbf{y}|\mathbf{x})$

Generative

Models that learn to generate new (input) samples that resemble the training data. $P(\mathbf{x})$

Discriminatory vs generative models



Discriminatory

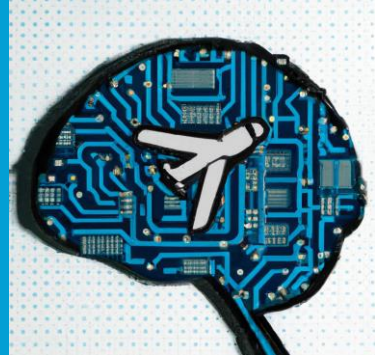
Models that learn to map input samples to target outputs, e.g., for classification or regression. $P(\mathbf{y}|\mathbf{x})$

Generative

Models that learn to generate new (input) samples that resemble the training data. $P(\mathbf{x})$

Learns without labels!

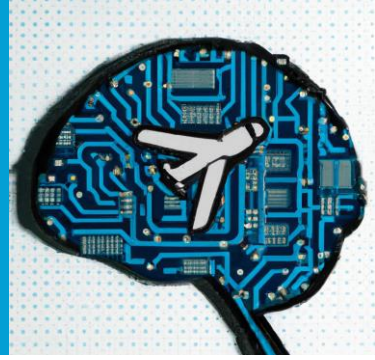
Generative AI



Very similar to “modeling” in the sciences: what are the underlying mechanisms that give rise to the observed data?



Generative AI



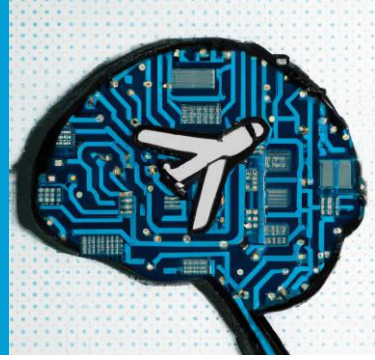
Very similar to “modeling” in the sciences: what are the underlying mechanisms that give rise to the observed data?

Even useful when you ultimately want to predict:

Compare predicting the weather from just examples of measurements and targets to learning an underlying generative model.

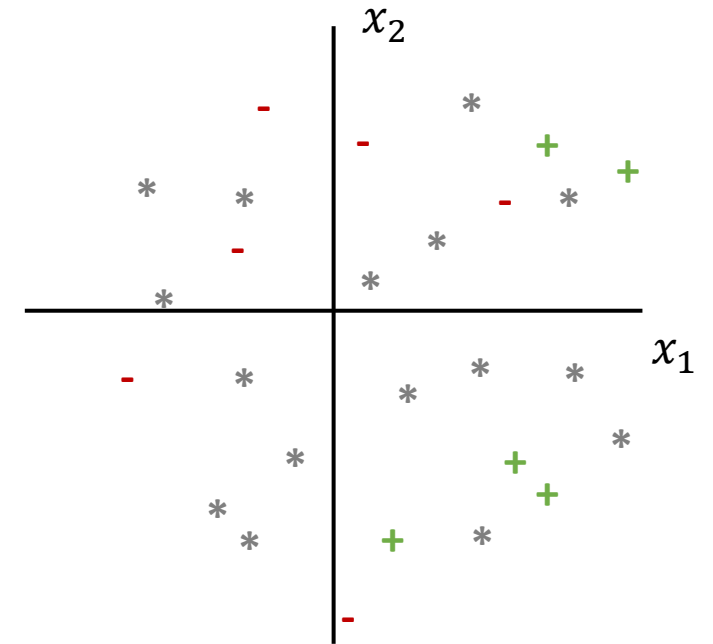


Generative models

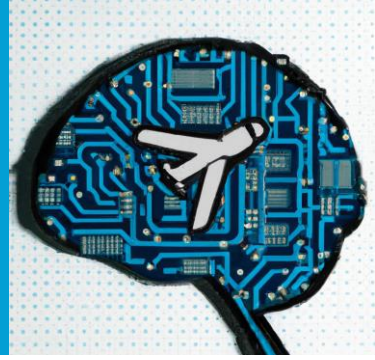


Advantages of generative models:

- Use domain knowledge by means of constraints in the process and differentiating important factors from noise

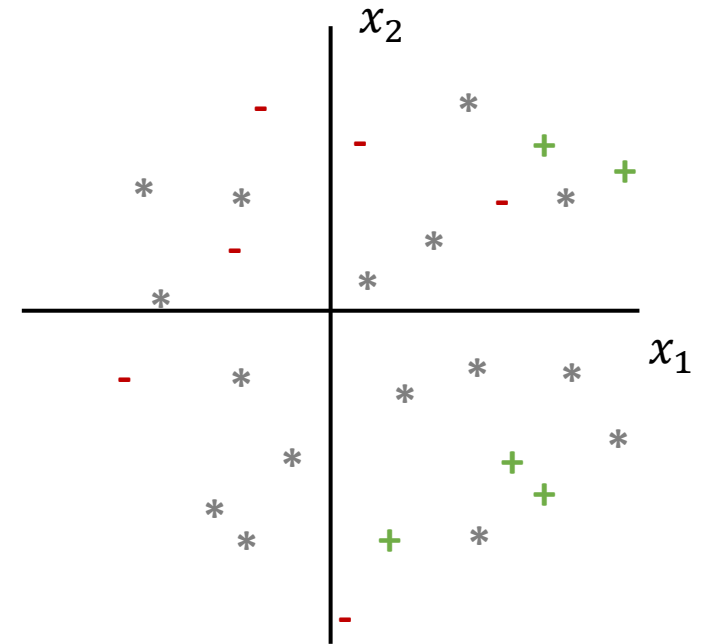


Generative models

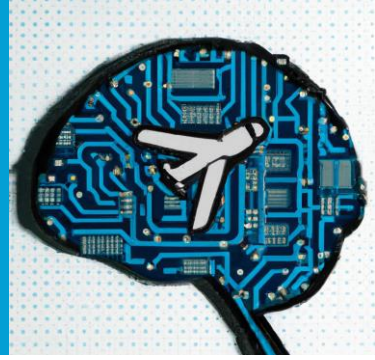


Advantages of generative models:

- Use domain knowledge by means of constraints in the process and differentiating important factors from noise
- Capture causal relations that generalize to unseen situations

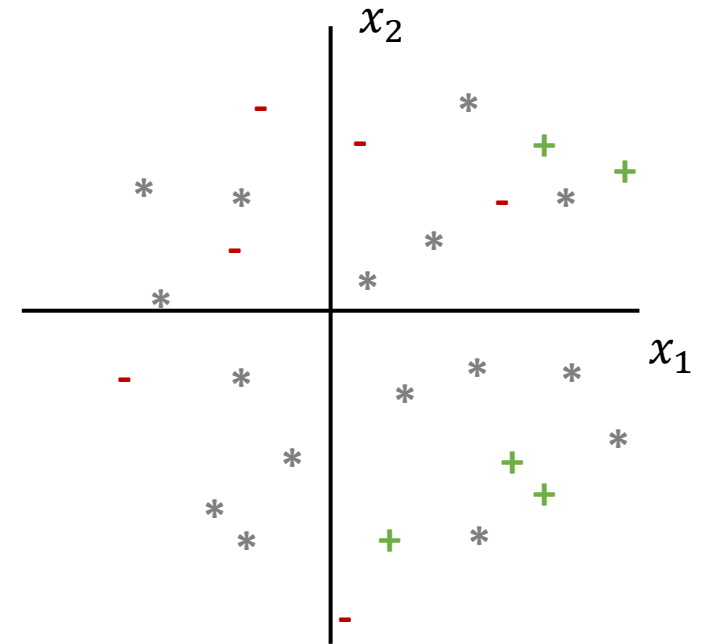


Generative models

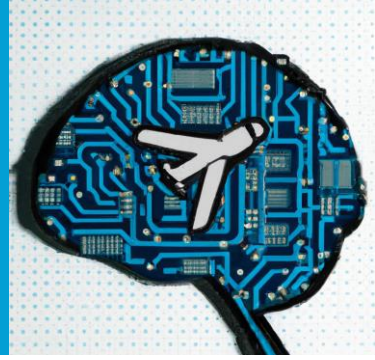


Advantages of generative models:

- Use domain knowledge by means of constraints in the process and differentiating important factors from noise
- Capture causal relations that generalize to unseen situations
- Build useful intermediate representations for downstream (discriminatory) tasks

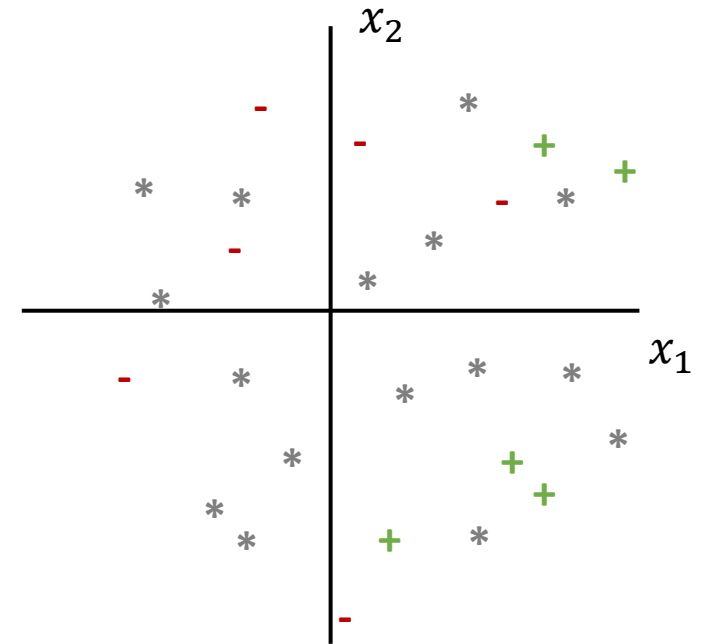


Generative models

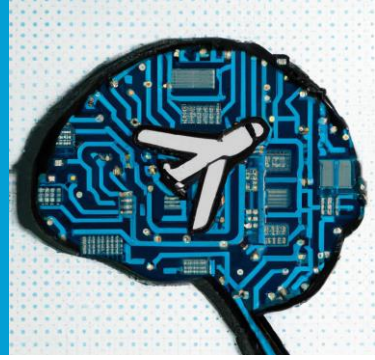


Advantages of generative models:

- Use domain knowledge by means of constraints in the process and differentiating important factors from noise
- Capture causal relations that generalize to unseen situations
- Build useful intermediate representations for downstream (discriminatory) tasks
- Expand the amount of data on which can be learned (e.g., in a semi-supervised setting)



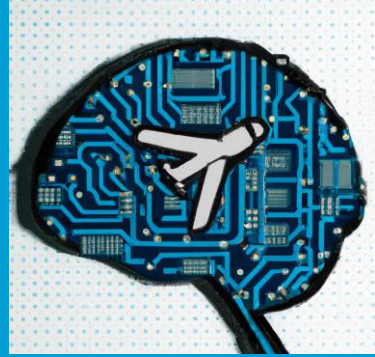
Generative AI approaches



Generative AI has been studied since at least the early 1980s, e.g., with “Boltzmann” machines, but it has blossomed with the arrival of deep learning.

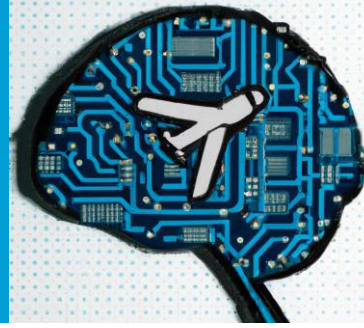
We will treat the following three approaches:

- Variational Auto-Encoders (VAEs)
- Generative Adversarial Networks (GANs)
- Diffusion models

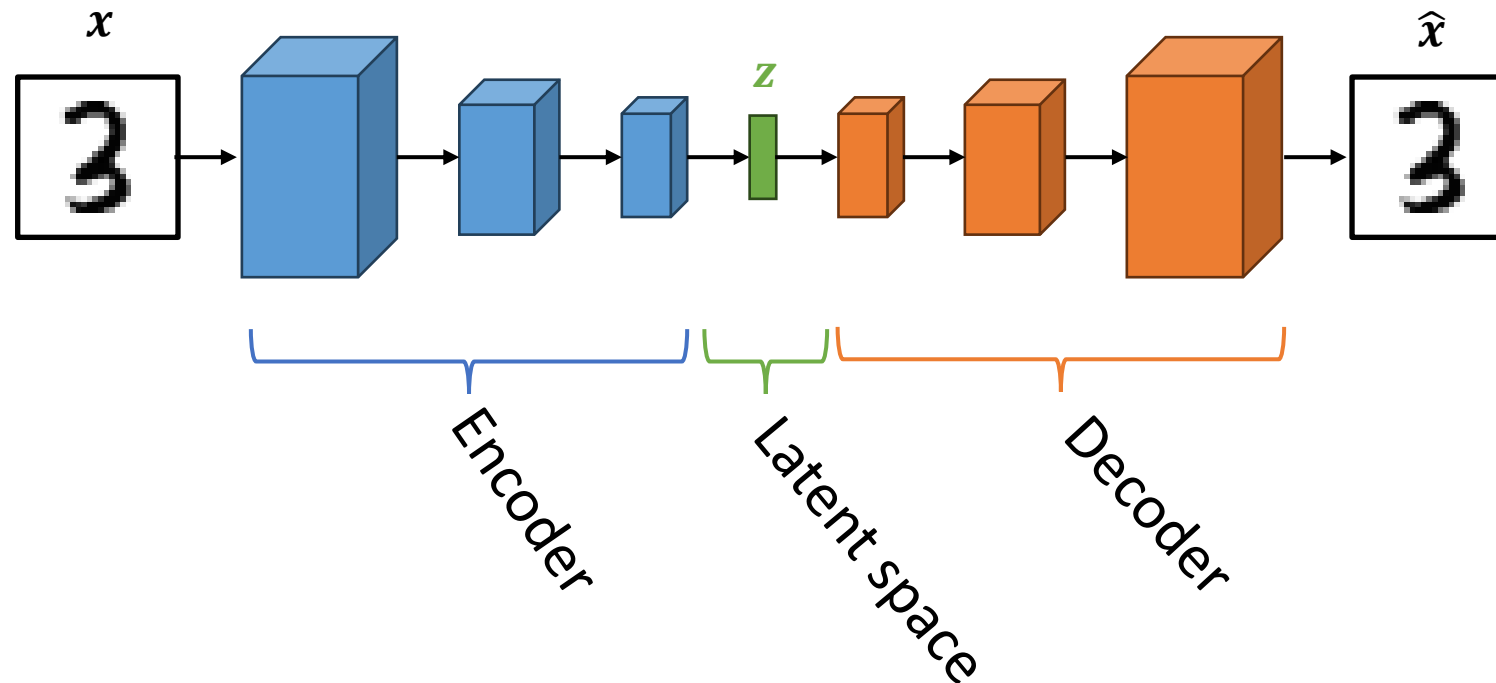


Variational Auto-encoder

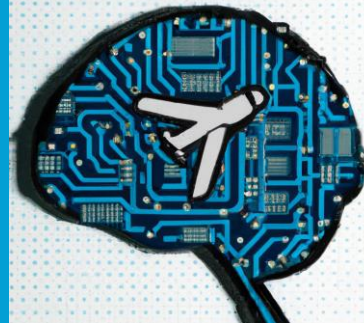
Auto-encoder



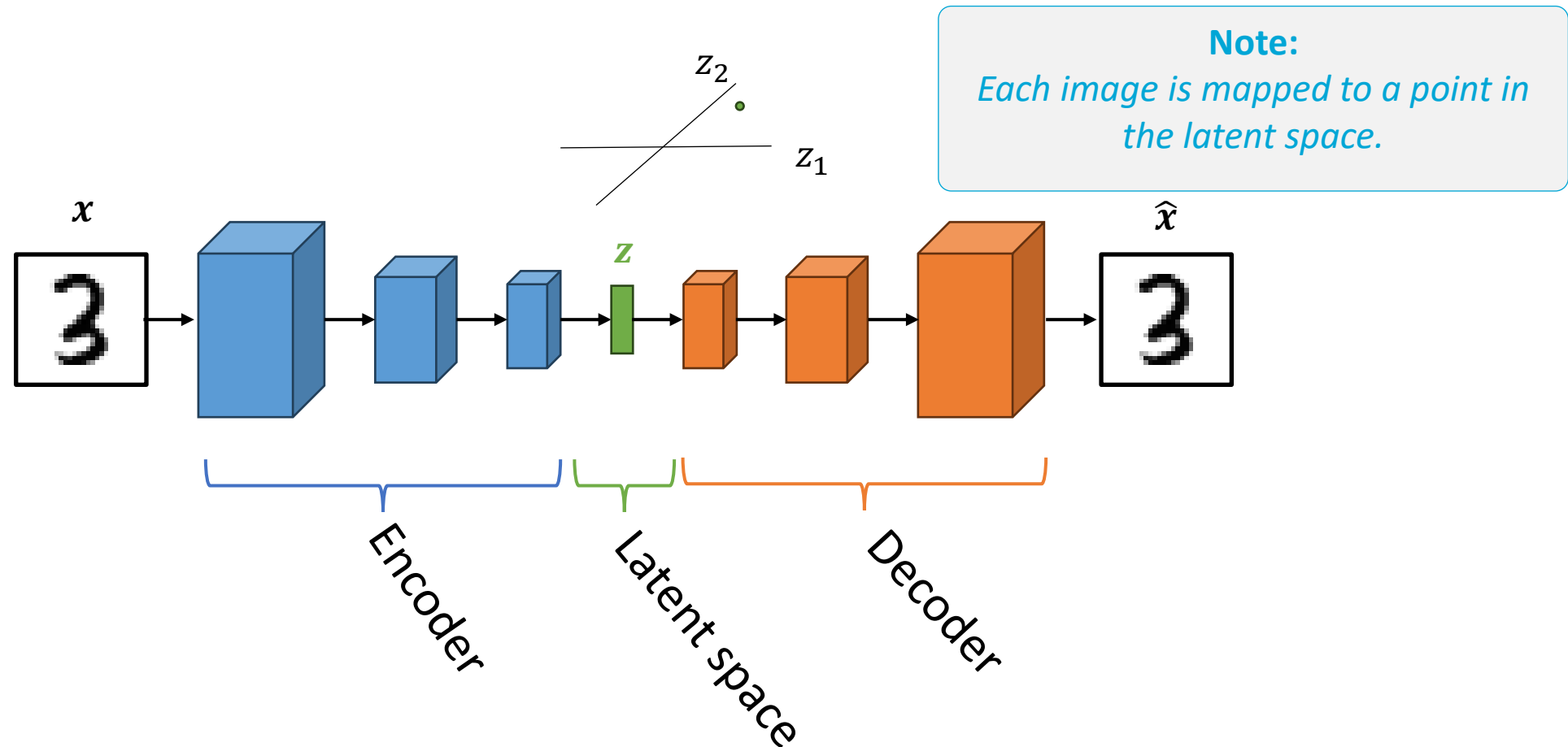
A standard auto-encoder fully convolutional network:



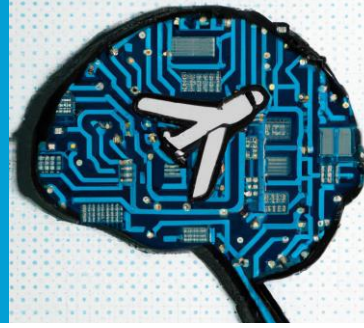
Auto-encoder



A standard auto-encoder fully convolutional network:

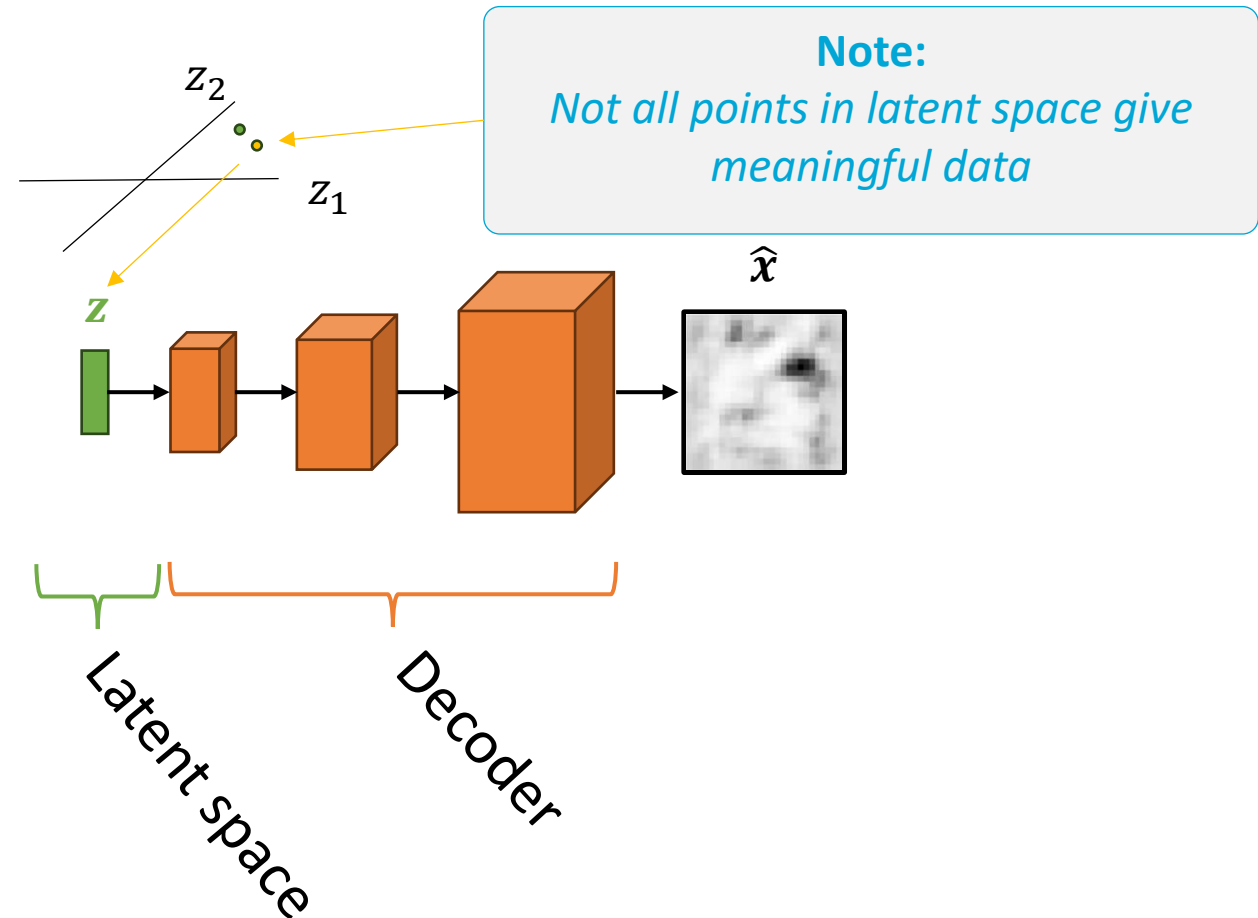


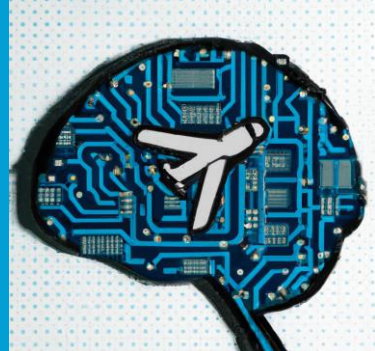
Can an auto-encoder generate new data?



Latent space is not continuous

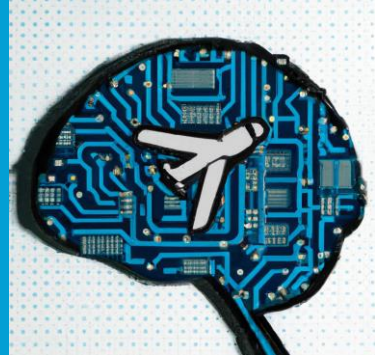
Latent space is not normalized



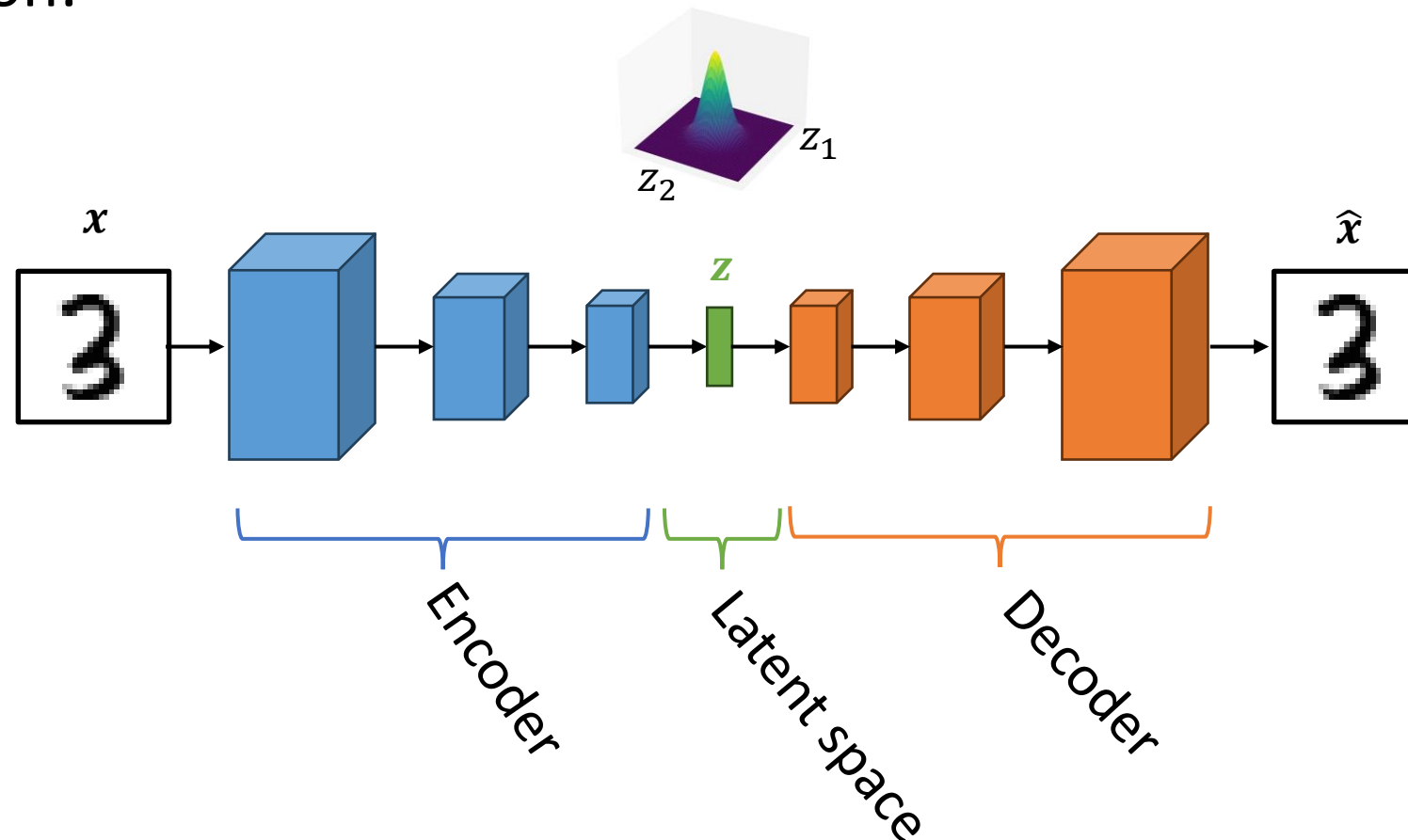


Solution?

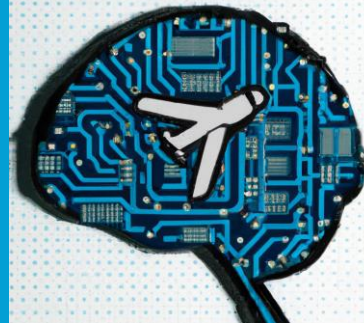
Variational auto-encoder



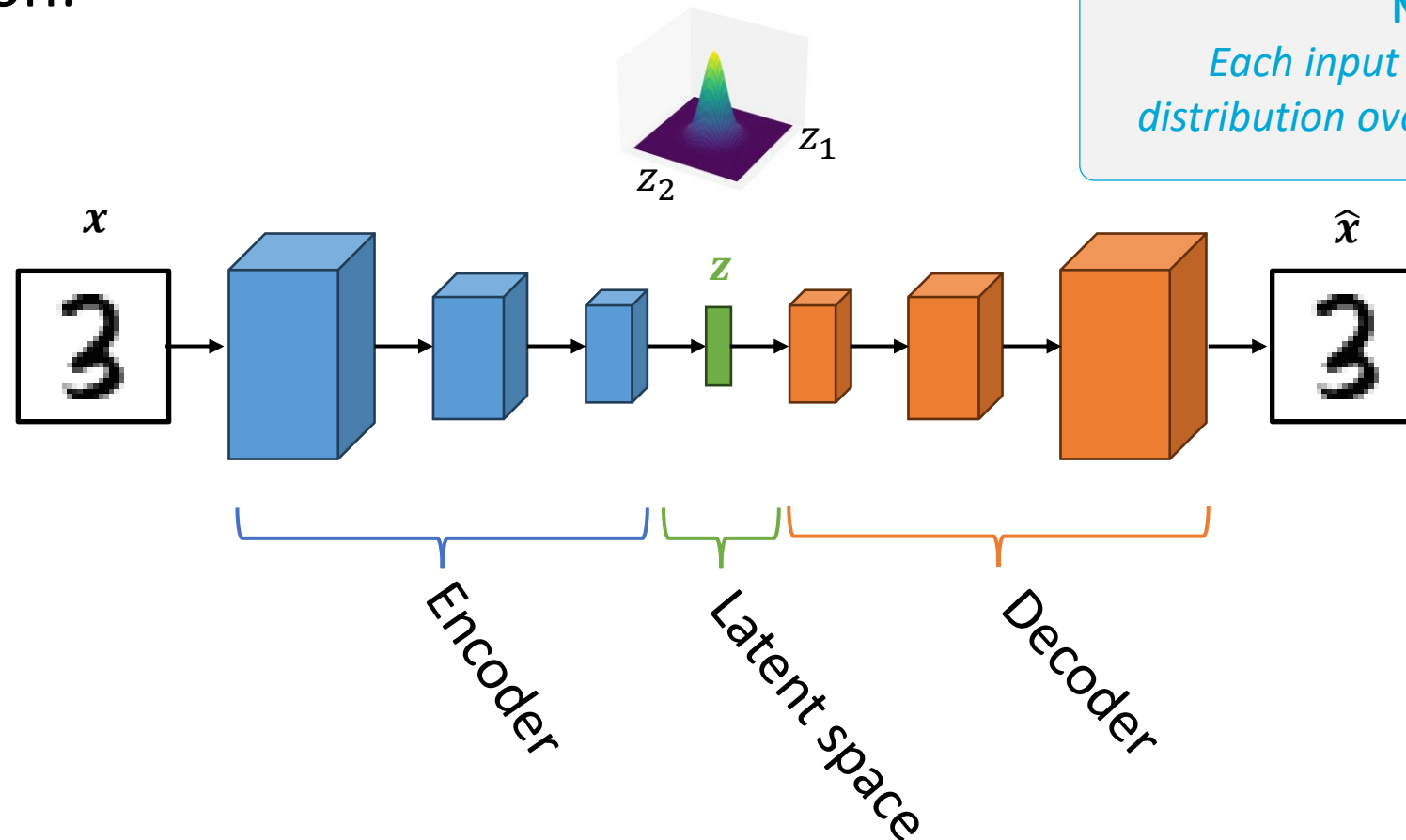
The core concept is to use a probability distribution as latent representation:



Variational auto-encoder

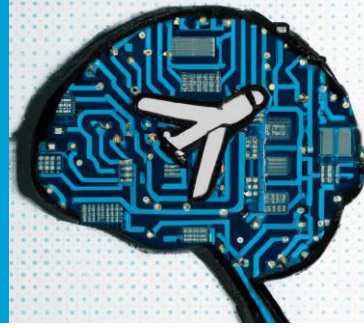


The core concept is to use a probability distribution as latent representation:

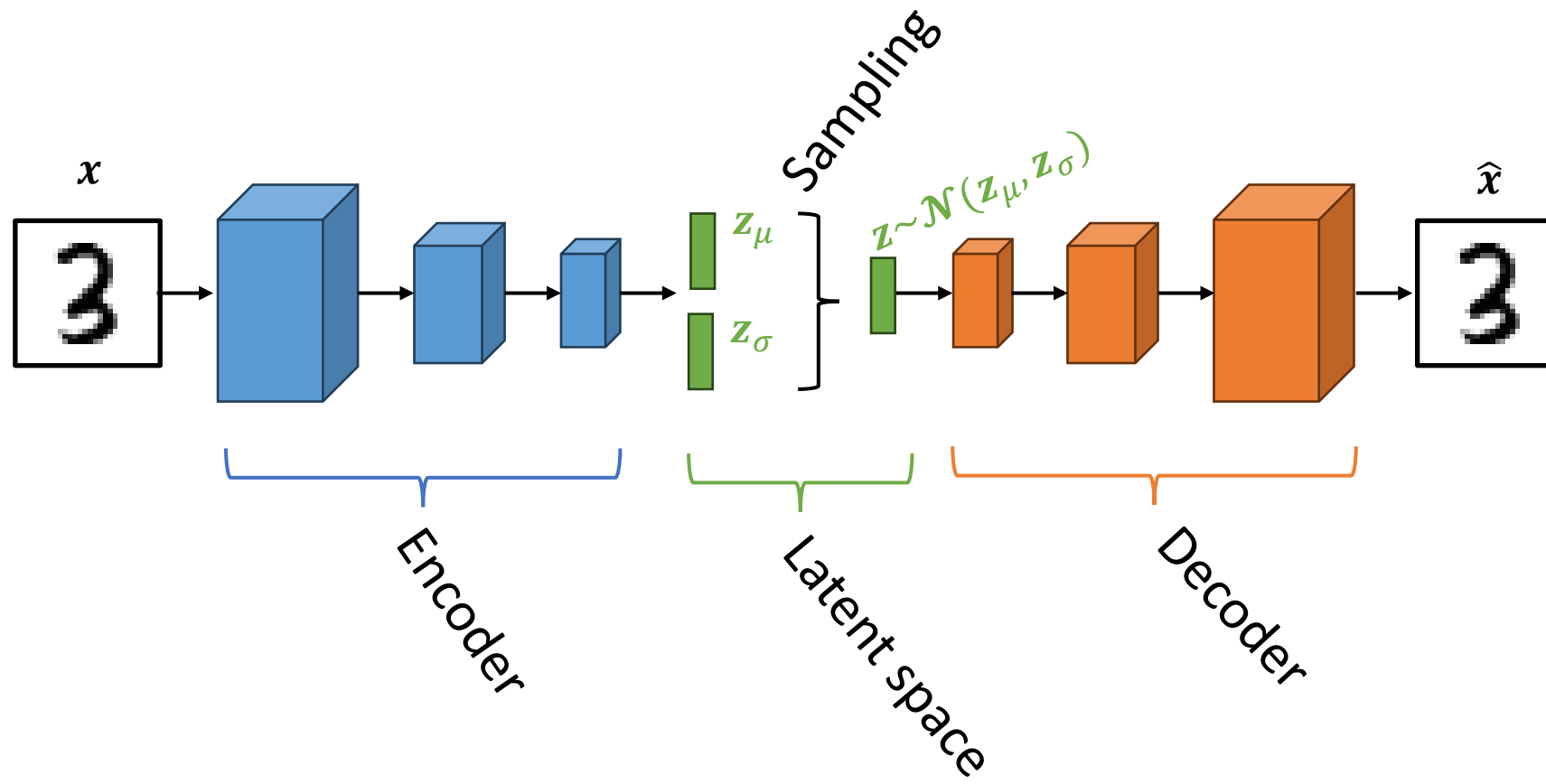


Note:
Each input is mapped to a distribution over the latent space.

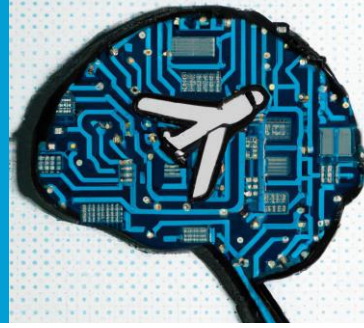
Variational auto-encoder



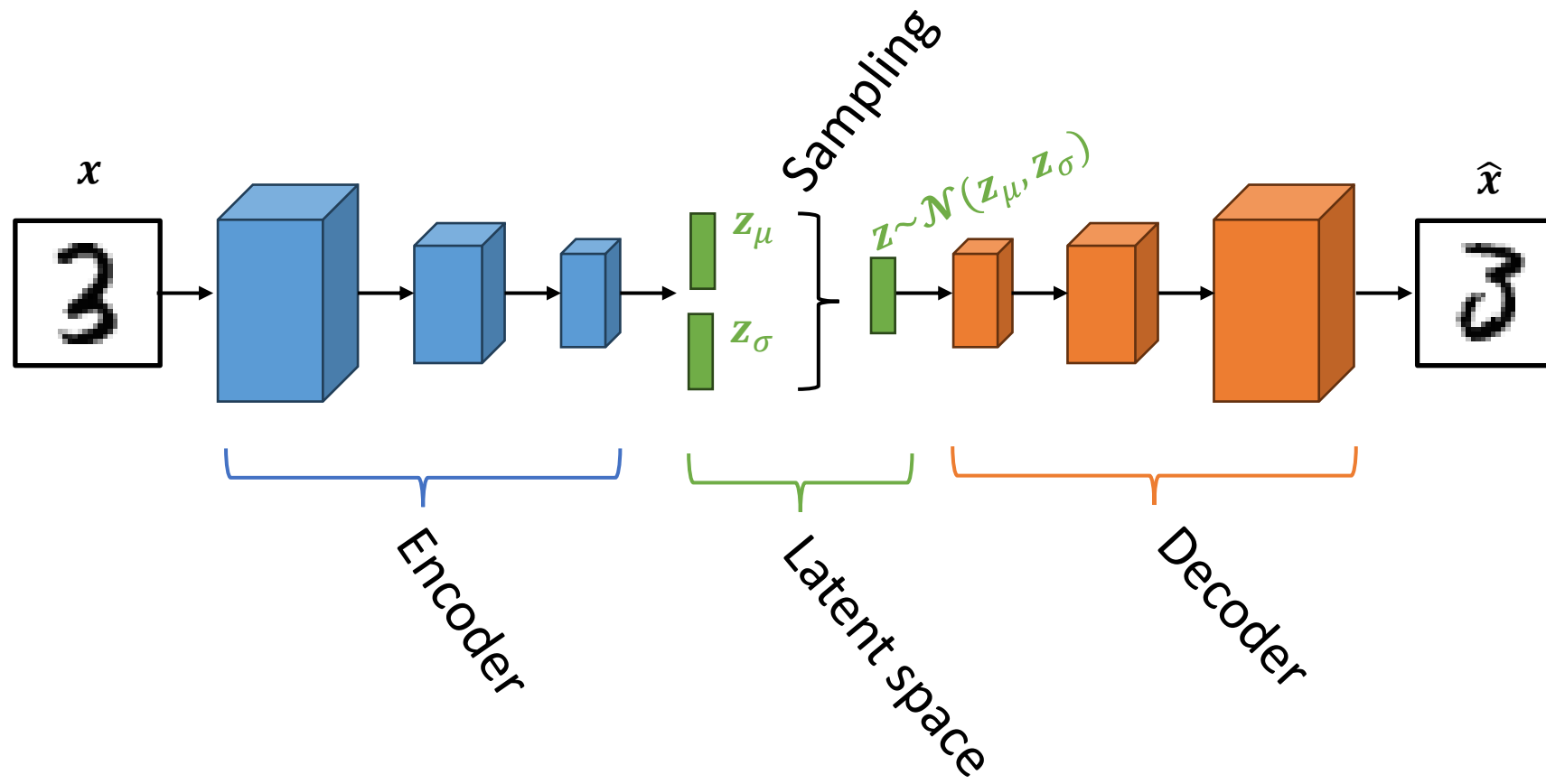
The core concept is to use a probability distribution as latent representation:



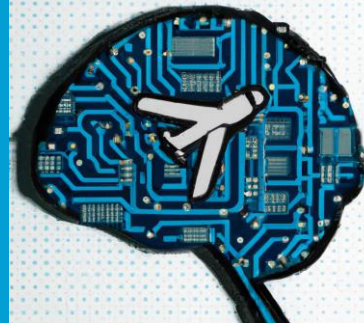
Variational auto-encoder



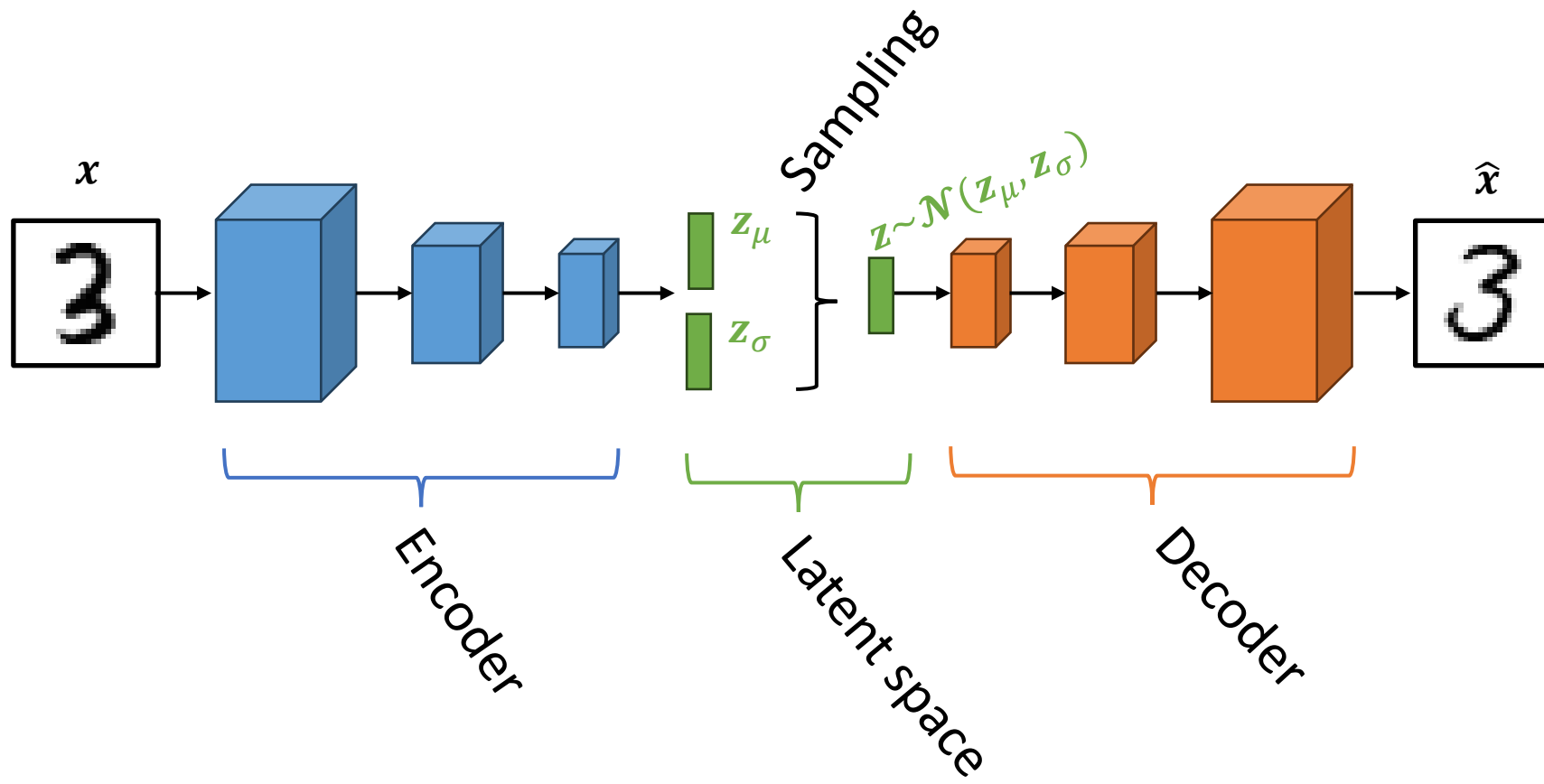
The core concept is to use a probability distribution as latent representation:



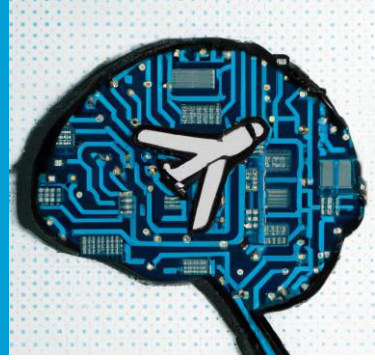
Variational auto-encoder



The core concept is to use a probability distribution as latent representation:



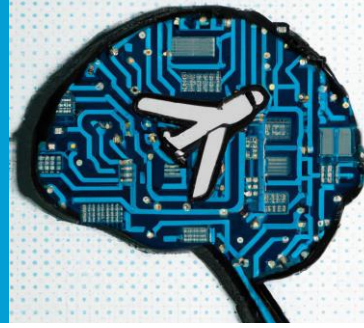
Variational auto-encoder



How does this network “learn” $P(\mathbf{x})$?

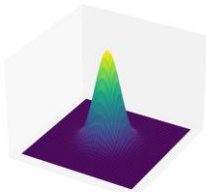
$$P(\mathbf{x}) = \int P(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \int P(\mathbf{x}|\mathbf{z})P(\mathbf{z}) d\mathbf{z}$$

Variational auto-encoder

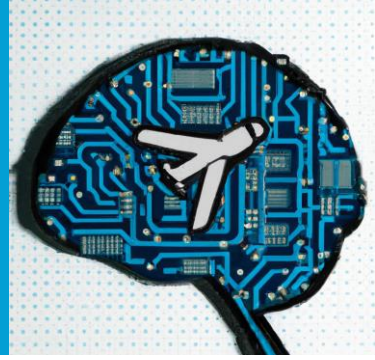


How does this network “learn” $P(\mathbf{x})$?

$$\begin{aligned} P(x) &= \int P(x, z) dz = \int P(x|z)P(z) dz \\ &= \int P(x|z)P(z) \frac{q(z|x)}{q(z|x)} dz = \mathbb{E}_q \left[\frac{P(x|z)P(z)}{q(z|x)} \right] \end{aligned}$$



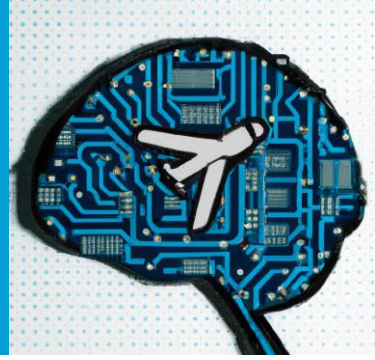
Variational auto-encoder



How does this network “learn” $P(\mathbf{x})$?

$$\begin{aligned} P(x) &= \int P(x, z) dz = \int P(x|z)P(z)dz \\ &= \int P(x|z)P(z) \frac{q(z|x)}{q(z|x)} dz = \mathbb{E}_q \left[\frac{P(x|z)P(z)}{q(z|x)} \right] \\ &\approx \frac{1}{N} \sum_{i=1}^N \frac{P(\hat{x}_i|z_i)P(z_i)}{q(z_i|x_i)} \end{aligned}$$

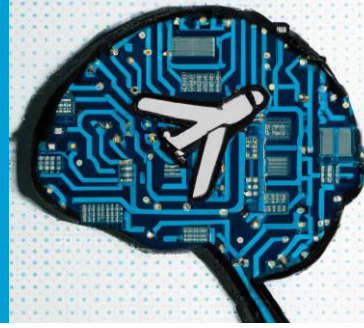
Variational auto-encoder



How does this network “learn” $P(\mathbf{x})$?

$$\begin{aligned} P(x) &= \int P(x, z) dz = \int P(x|z)P(z)dz \\ &= \int P(x|z)P(z) \frac{q(z|x)}{q(z|x)} dz = \mathbb{E}_q \left[\frac{P(x|z)P(z)}{q(z|x)} \right] \\ &\approx \frac{1}{N} \sum_{i=1}^N \frac{P(\hat{x}_i|z_i)P(z_i)}{q(z_i|x_i)} \end{aligned}$$

Variational auto-encoder

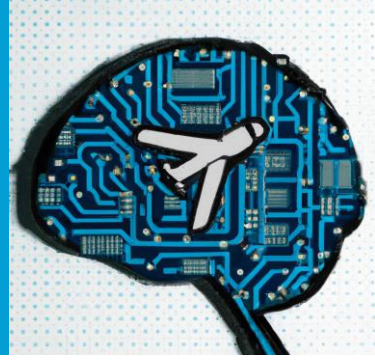


How does this network “learn” $P(\mathbf{x})$?

$$\begin{aligned} P(x) &= \int P(x, z) dz = \int P(x|z)P(z)dz \\ &= \int P(x|z)P(z) \frac{q(z|x)}{q(z|x)} dz = \mathbb{E}_q \left[\frac{P(x|z)P(z)}{q(z|x)} \right] \\ &\approx \frac{1}{N} \sum_{i=1}^N \frac{P(\hat{x}_i|z_i)P(z_i)}{q(z_i|x_i)} \end{aligned}$$

→ **Prior** parameters for the latent space distribution are chosen by the designer.

Variational auto-encoder

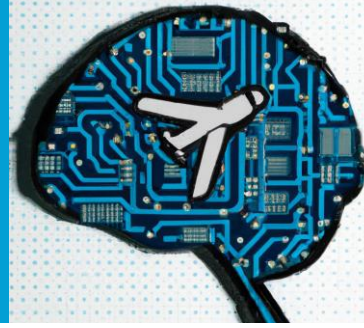


How does this network “learn” $P(\mathbf{x})$?

$$\begin{aligned} P(x) &= \int P(x, z) dz = \int P(x|z)P(z)dz \\ &= \int P(x|z)P(z) \frac{q(z|x)}{q(z|x)} dz = \mathbb{E}_q \left[\frac{P(x|z)P(z)}{q(z|x)} \right] \\ &\approx \frac{1}{N} \sum_{i=1}^N \frac{P(\hat{x}_i|z_i)P(z_i)}{q(z_i|x_i)} \end{aligned}$$

These encoder and decoder functions are represented with **neural networks**, and their parameters are **trained** for the task, to maximize the likelihood.

Variational auto-encoder

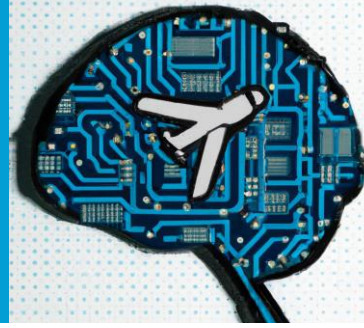


How does this network “learn” $P(\mathbf{x})$?

$$\begin{aligned} P(x) &= \int P(x, z) dz = \int P(x|z)P(z)dz \\ &= \int P(x|z)P(z) \frac{q(z|x)}{q(z|x)} dz = \mathbb{E}_q \left[\frac{P(x|z)P(z)}{q(z|x)} \right] \\ &\approx \frac{1}{N} \sum_{i=1}^N \frac{P(\hat{x}_i|z_i)P(z_i)}{q(z_i|x_i)} \end{aligned}$$

Optimally, $q^*(z|x) = P(z|x)$.

Training of a VAE

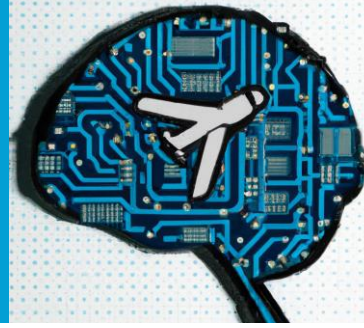


The training optimizes the probability of $\mathbf{x}$, given the parameters Θ :

$$\max_{\Theta} P(\mathbf{x}; \Theta) = \max_{\Theta} (\log(\mathbb{E}_q \left[\frac{P(\hat{\mathbf{x}}|z, \Theta_d)P(z)}{q(z|\mathbf{x}, \Theta_e)} \right]))$$

Where Θ_d are the parameters of the decoder and Θ_e of the encoder.

Training of a VAE



The training optimizes the probability of $\mathbf{x}$, given the parameters Θ :

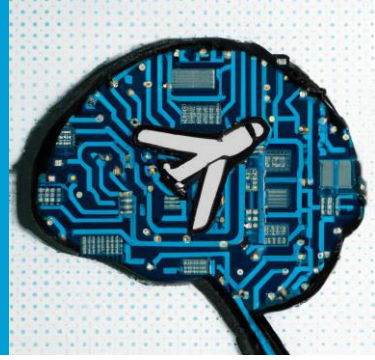
$$\max_{\Theta} P(\mathbf{x}; \Theta) = \max_{\Theta} (\log(\mathbb{E}_q \left[\frac{P(\hat{\mathbf{x}}|z, \Theta_d)P(z)}{q(z|\mathbf{x}, \Theta_e)} \right]))$$

Where Θ_d are the parameters of the decoder and Θ_e of the encoder.

Furthermore, by Jensen's inequality:

$$\log \left(\mathbb{E}_q \left[\frac{P(\mathbf{x}|z, \Theta_d)P(z)}{q(z|\mathbf{x}, \Theta_e)} \right] \right) \geq \mathbb{E}_q \left[\log \left(\frac{P(\mathbf{x}|z, \Theta_d)P(z)}{q(z|\mathbf{x}, \Theta_e)} \right) \right] = \text{ELBO}(\Theta)$$

Training of a VAE

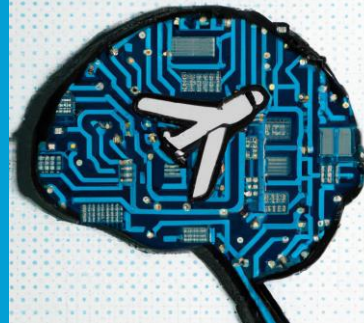


Maximizing this latter function, we get two terms:

$$\text{ELBO}(\Theta) = \sum_{i=1}^N \mathbb{E}_q[P(x|z, \Theta_d)] - D_{KL}(q(z|x, \Theta_e) \parallel P(z))$$

Where D_{KL} is the [Kullback-Leibler divergence](#), which captures how similar two probability distributions are (zero when identical).

Training of a VAE

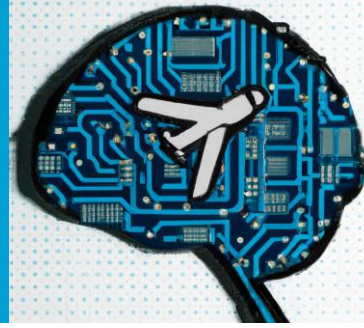


Maximizing this latter function, we get two terms:

$$\text{ELBO}(\Theta) = \underbrace{\sum_{i=1}^N \mathbb{E}_q[P(x|z, \Theta_d)]}_{\text{Reconstruction}} - \underbrace{D_{KL}(q(z|x, \Theta_e) \parallel P(z))}_{\text{Structure of the latent space}}$$

Where D_{KL} is the [Kullback-Leibler divergence](#), which captures how similar two probability distributions are (zero when identical).

Loss function

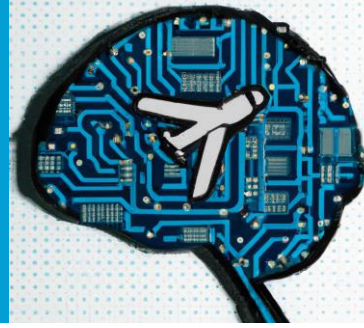


For a desired Gaussian prior distribution of $\mathcal{N}(\mathbf{0}, \mathbf{I})$, the Kullback-Leibler divergence leads to the loss:

$$\mathcal{L}_{\text{KL}} = \sum_{i=1}^D \frac{1}{2} \left(\log \left(\frac{1}{\sigma_i^2} \right) + \sigma_i^2 + \mu_i^2 - 1 \right)$$

Where $\mathbf{z}$ has D dimensions.

Loss function

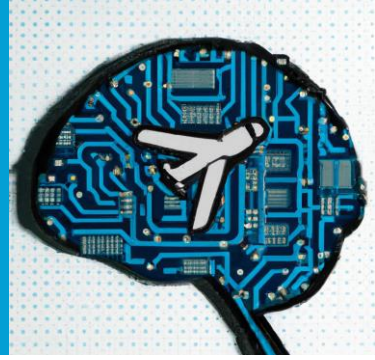


For the reconstruction term, the maximal likelihood loss function is typically used:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{M} \sum_{i=1}^M (x_i - \hat{x}_i)^2$$

Where $\hat{x}_i$ is a reconstructed variable, for instance a pixel in an image.

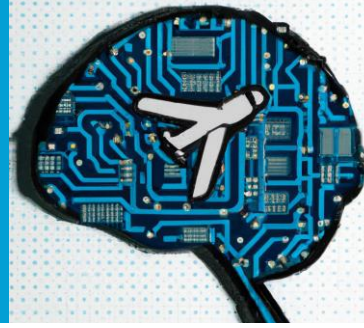
Loss function



The complete loss function then is:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{KL}}$$

Loss function

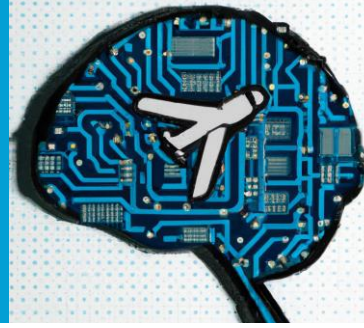


The complete loss function then is:

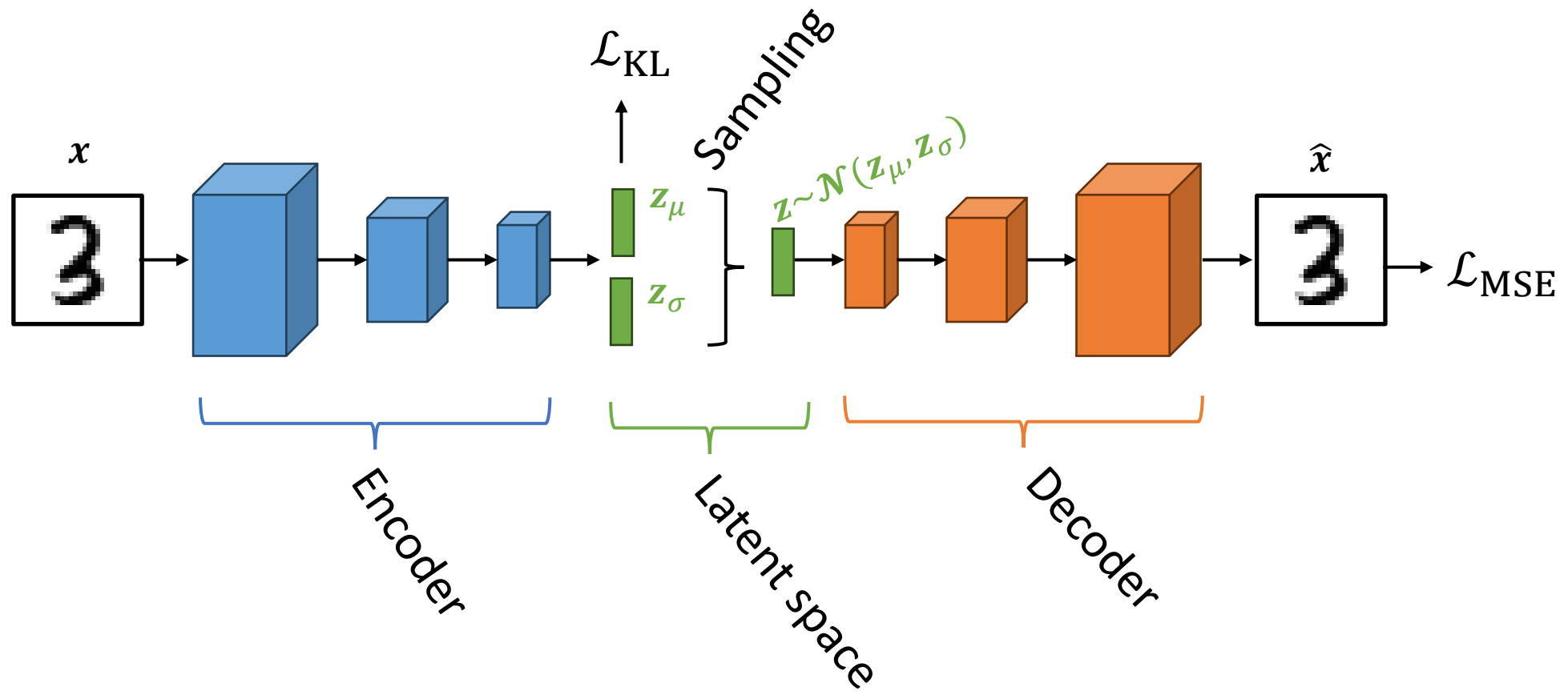
$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \beta \mathcal{L}_{\text{KL}}$$

A Lagrangian multiplier β can be used to trade off reconstruction quality with the divergence from the prior (β -VAE).

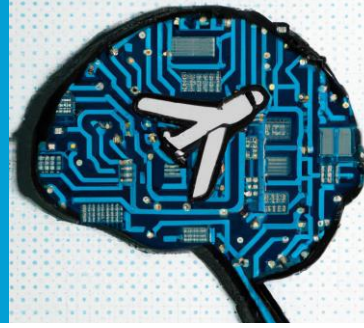
Training of a VAE



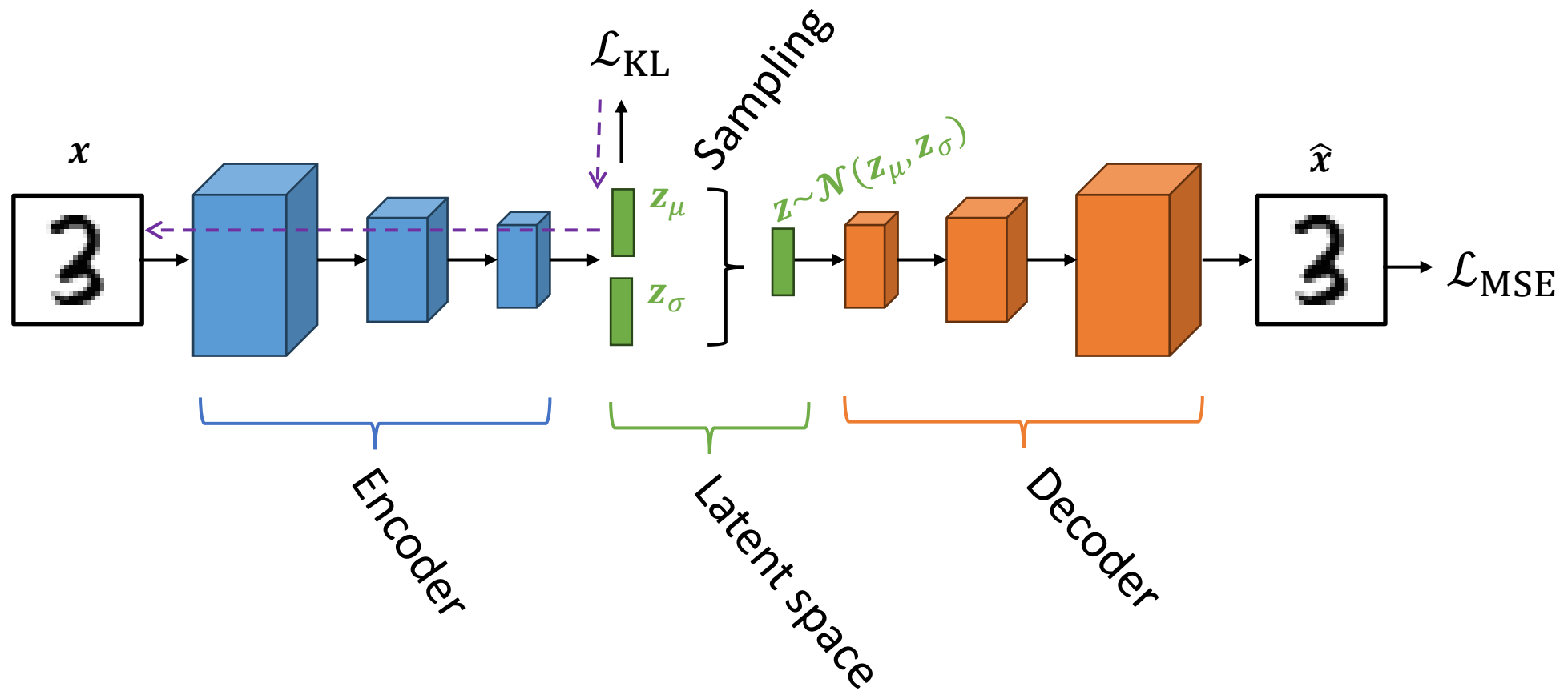
How to perform backpropagation through the random vector $\mathbf{z}$?



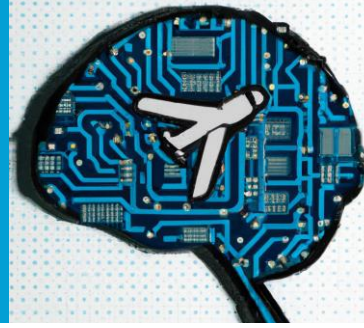
Training of a VAE



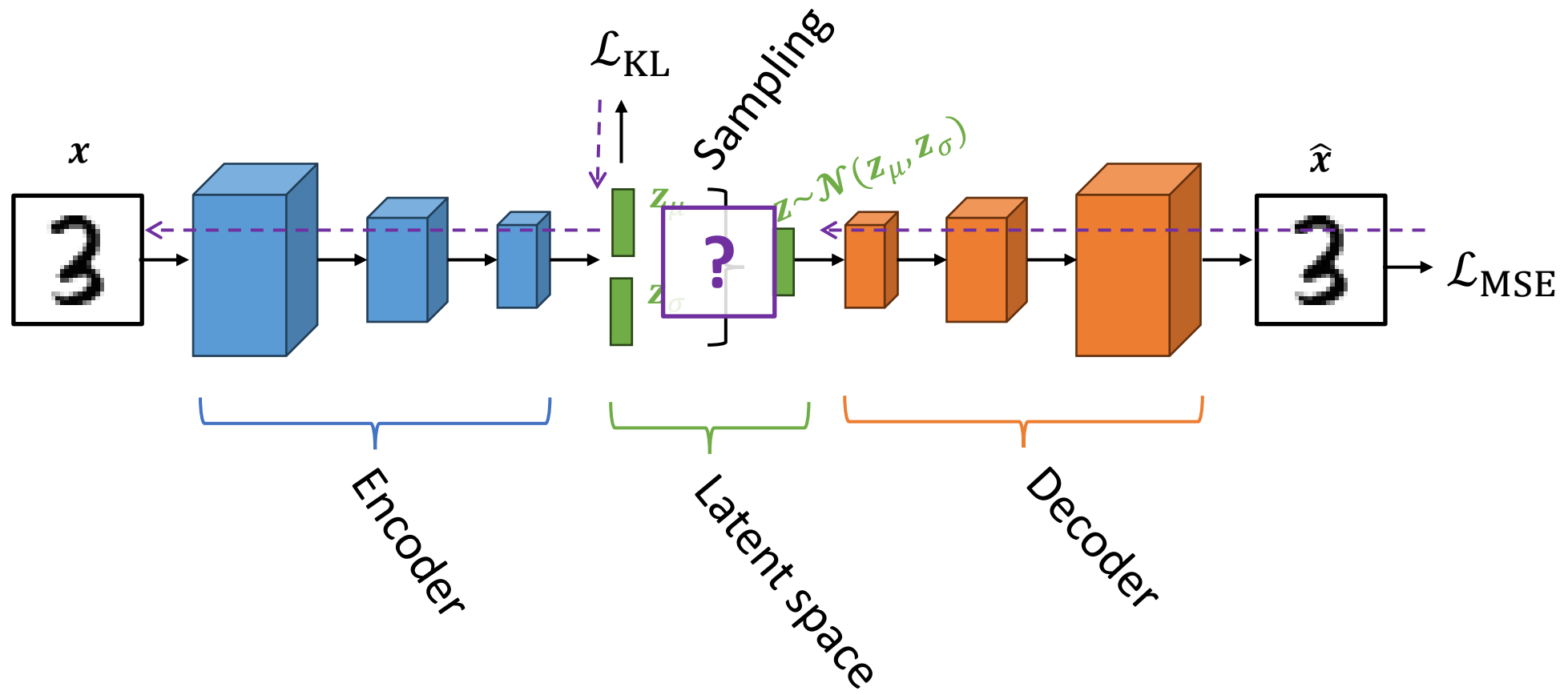
How to perform backpropagation through the random vector $\mathbf{z}$?



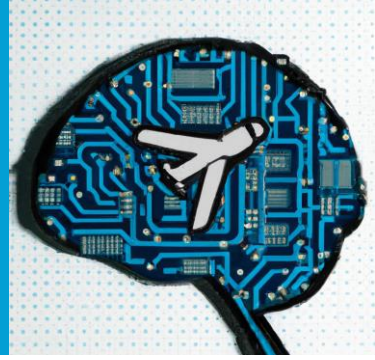
Training of a VAE



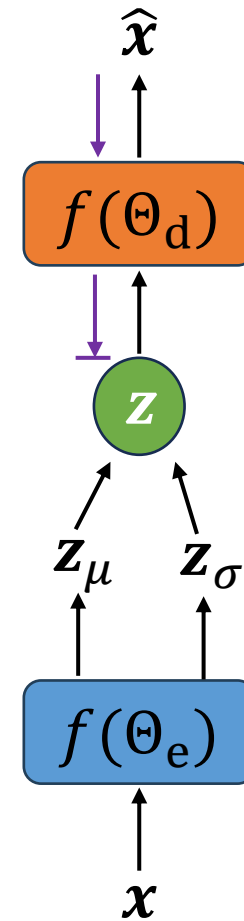
How to perform backpropagation through the random vector $\mathbf{z}$?



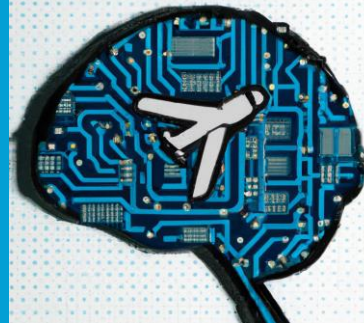
Reparameterization trick



If we sample $\mathbf{z}$ directly, backpropagation will stop at $\mathbf{z}$.



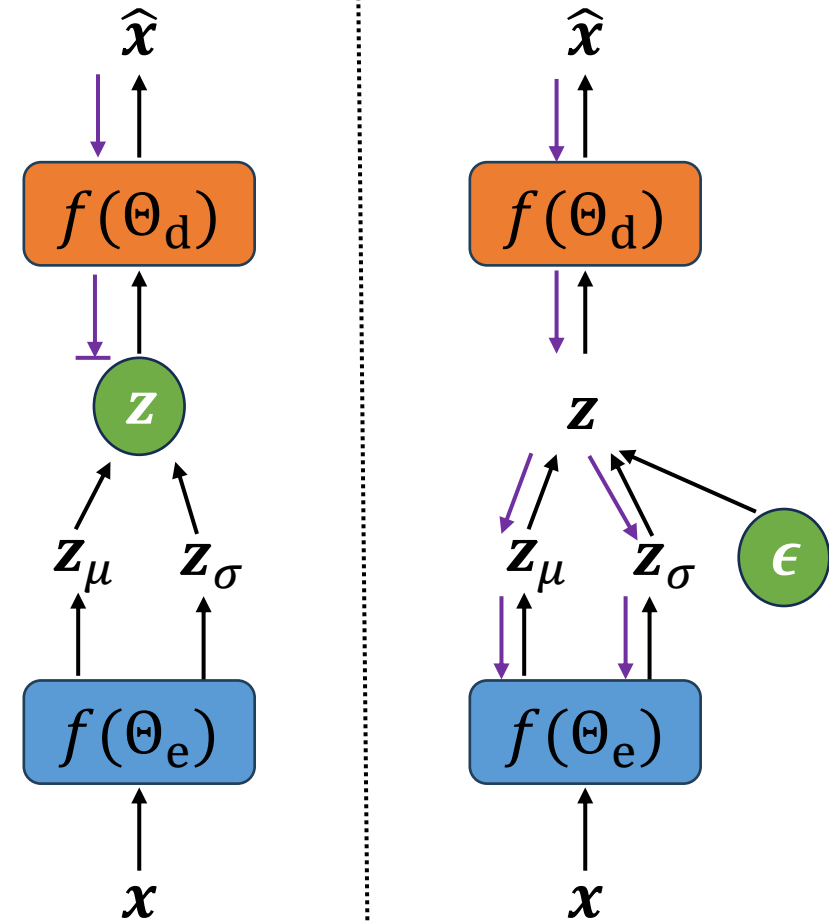
Reparameterization trick



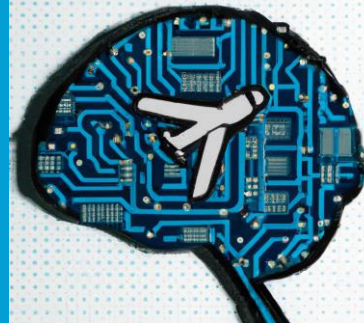
As a solution, reformulate $\mathbf{z}$ as:

$$\mathbf{z} = \mathbf{z}_\mu + \mathbf{z}_\sigma \odot \boldsymbol{\epsilon}$$

Where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\odot$ is element-wise multiplication. Then backpropagate through $\mathbf{z}_\mu$ and $\mathbf{z}_\sigma$.



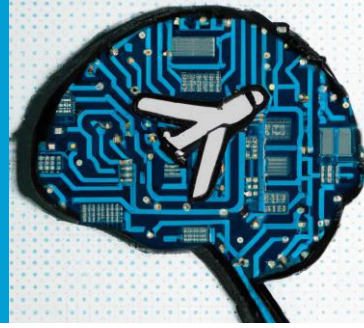
Results



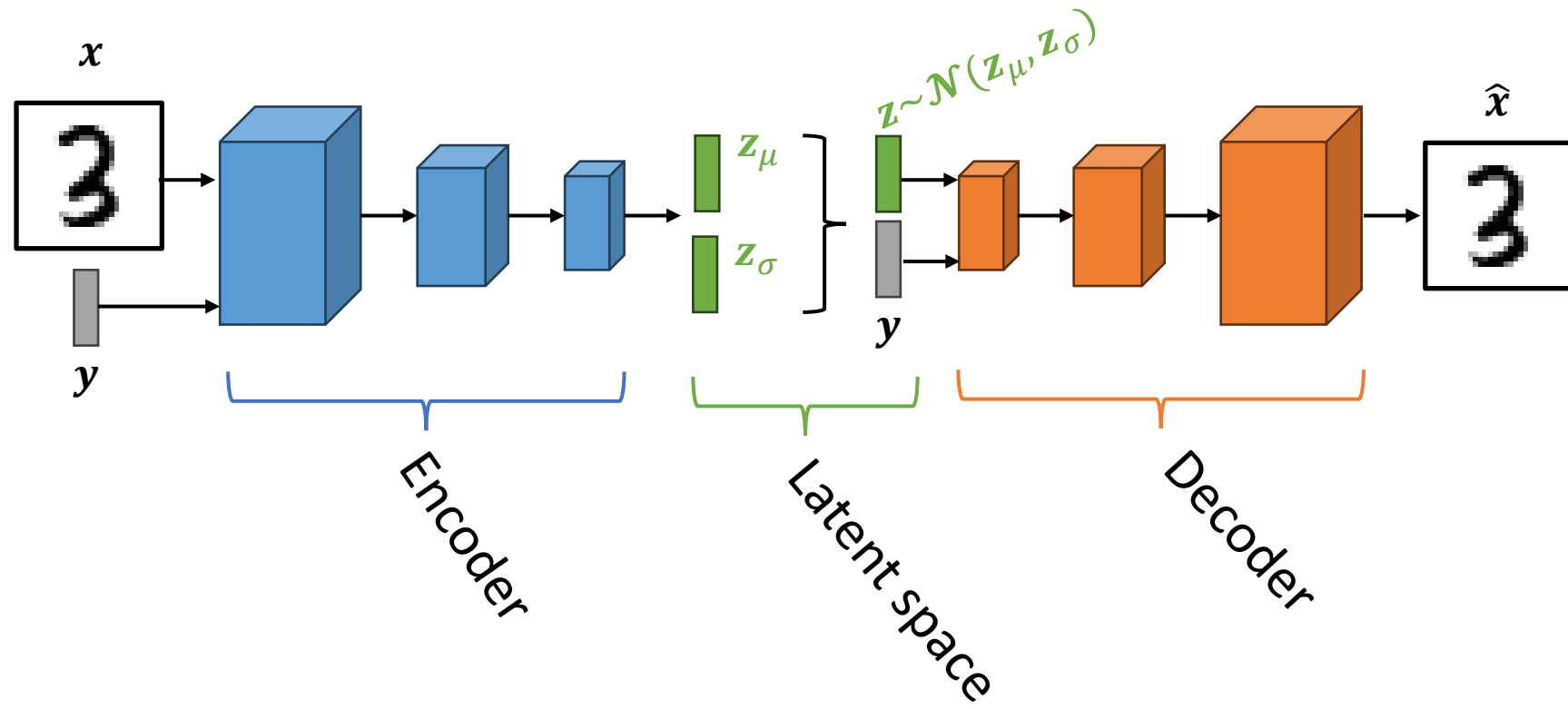
Sampling random paths in 512-dimensional latent space of a VAE, which was trained on Labeled Faces in the Wild (LFW) dataset.



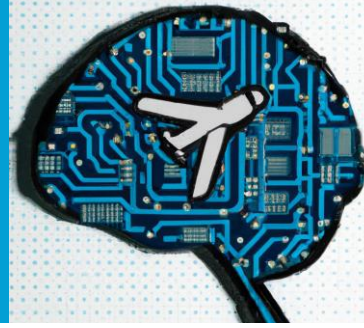
Conditional VAE



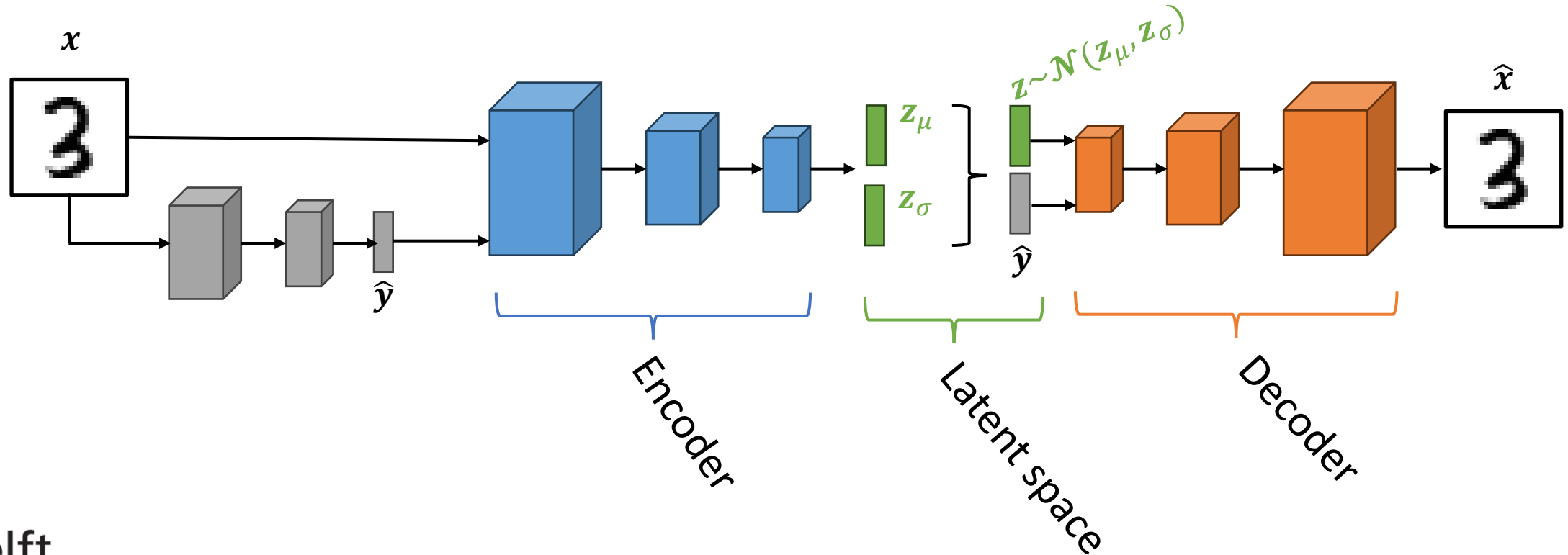
Condition the networks on additional information $\mathbf{y}$, e.g., encoding the image class with one-hot encoding.



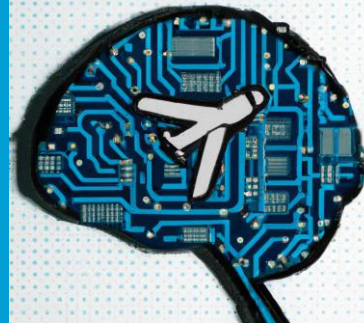
Conditional VAE



Condition the networks on additional information $\mathbf{y}$, e.g., encoding the image class with one-hot encoding.

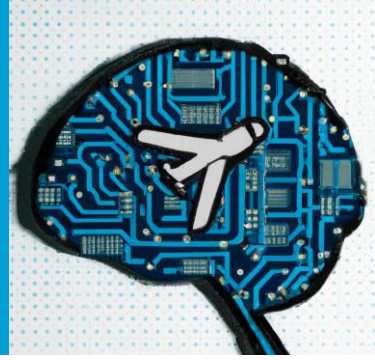


VAE challenges



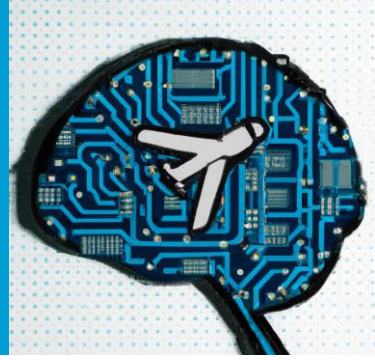
- Inexpressive “posterior” $q(z|x)$ - e.g., limited to independent Gaussian distribution
 - Work out the math for more complex distributions (e.g., full Gaussian)
 - Use additional (“auxiliary”) latent variables
 - “Normalizing flow”: Composing a complex distribution from a sequence of smaller steps

VAE challenges



- Inexpressive “posterior” $q(z|x)$ - e.g., limited to independent Gaussian distribution
 - Work out the math for more complex distributions (e.g., full Gaussian)
 - Use additional (“auxiliary”) latent variables
 - “Normalizing flow”: Composing a complex distribution from a sequence of smaller steps
- Posterior “collapse”, i.e., $q(z|x) = P(z)$
 - Use β -VAE, potentially varying β during training

VAEs



Advantages:



...



...

Disadvantages:

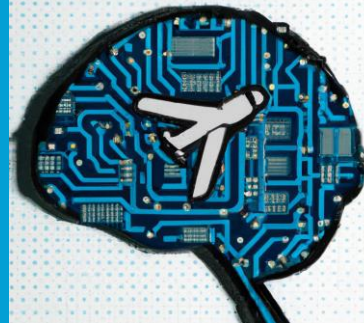


...



...

VAEs

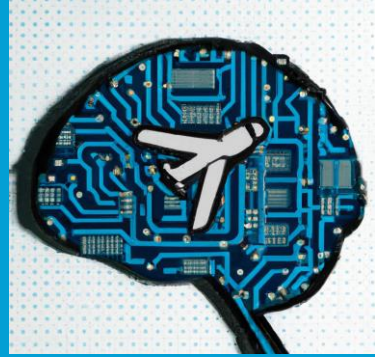


Advantages:

- ✓ Simple
- ✓ Easy to train

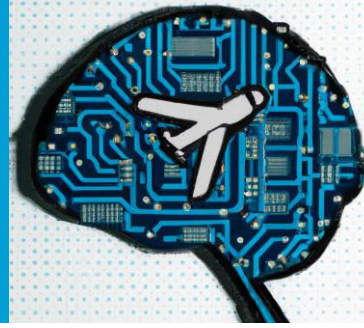
Disadvantages:

- ✗ Posterior “collapse” -> Blurry
- ✗ Inexpressive “posterior” all data fit a normally distributed latent space

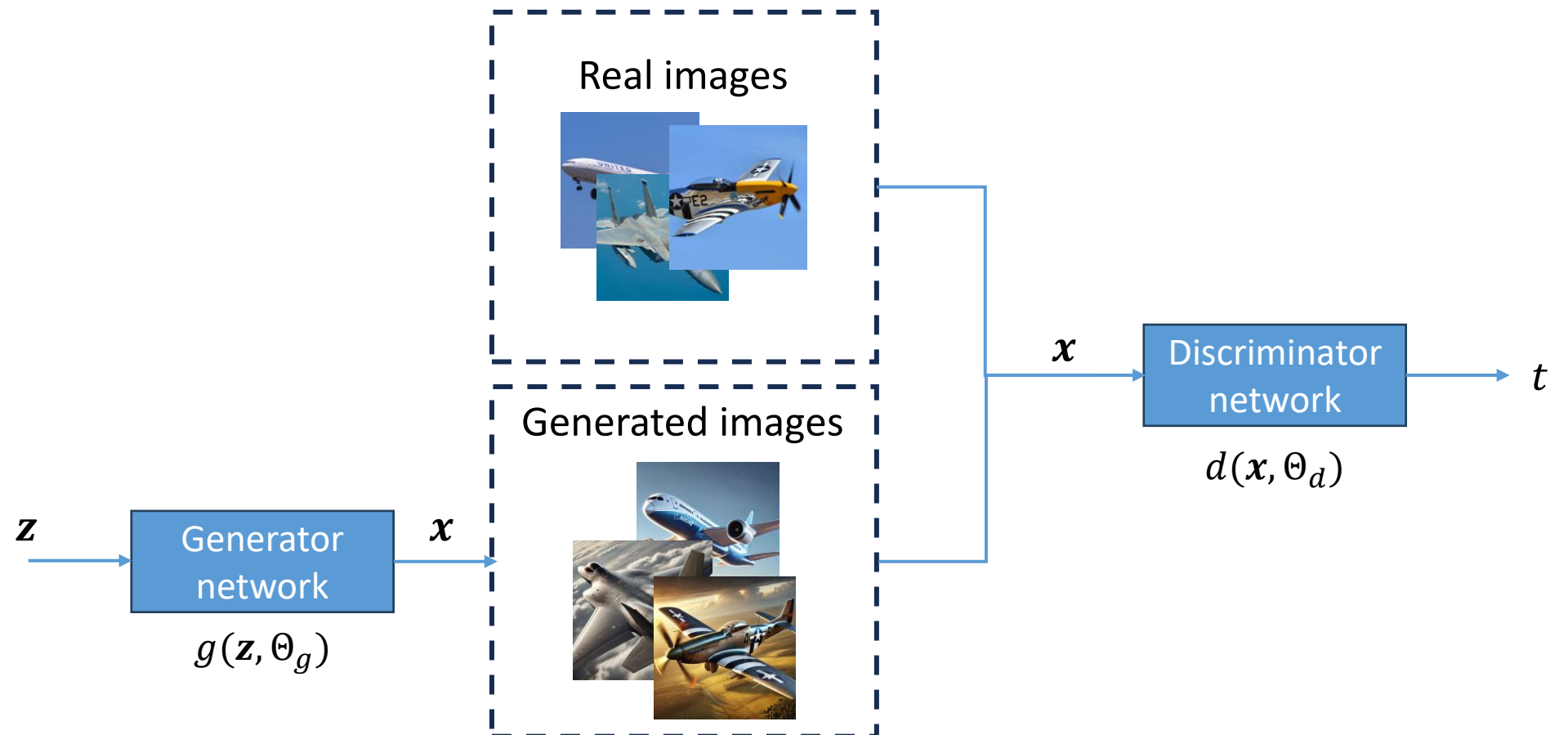


Generative Adversarial Networks

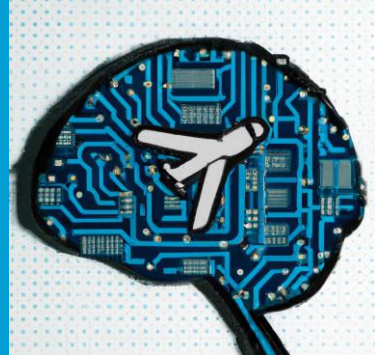
Central idea



Train a **generator** and **discriminator** network:



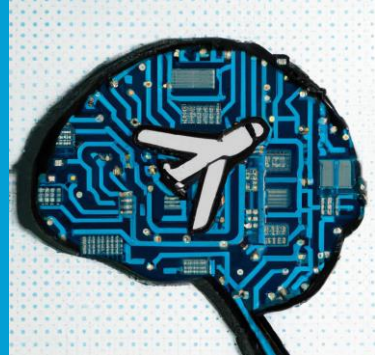
Adversarial training



Zero-sum game between generator and discriminator.

The discriminator needs to classify an image as real (1) or “fake” (0).

Adversarial training



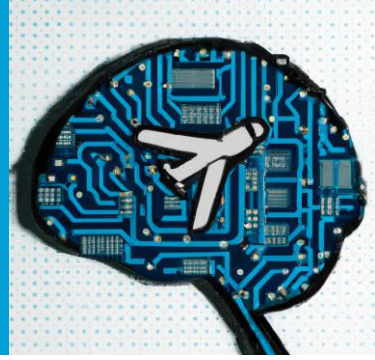
Zero-sum game between generator and discriminator.

The discriminator needs to classify an image as real (1) or “fake” (0).

We can use a standard cross-entropy loss:

$$E = -\frac{1}{N} \sum_{i=1}^N (t_i \ln(d(\mathbf{x}_i)) + (1 - t_i) \ln(1 - d_i))$$

Adversarial training



Zero-sum game between generator and discriminator.

The discriminator needs to classify an image as real (1) or “fake” (0).

We can use a standard cross-entropy loss:

$$E = -\frac{1}{N} \sum_{i=1}^N (t_i \ln(d(\mathbf{x}_i)) + (1 - t_i) \ln(1 - d_i))$$

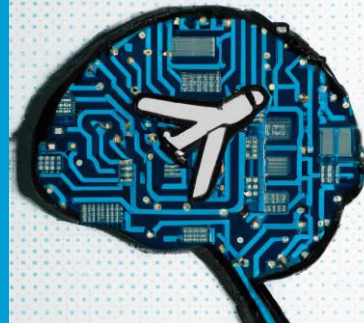
The updates for the discriminator are:

$$\Delta \Theta_d = -\alpha \nabla_{\Theta_d} E$$

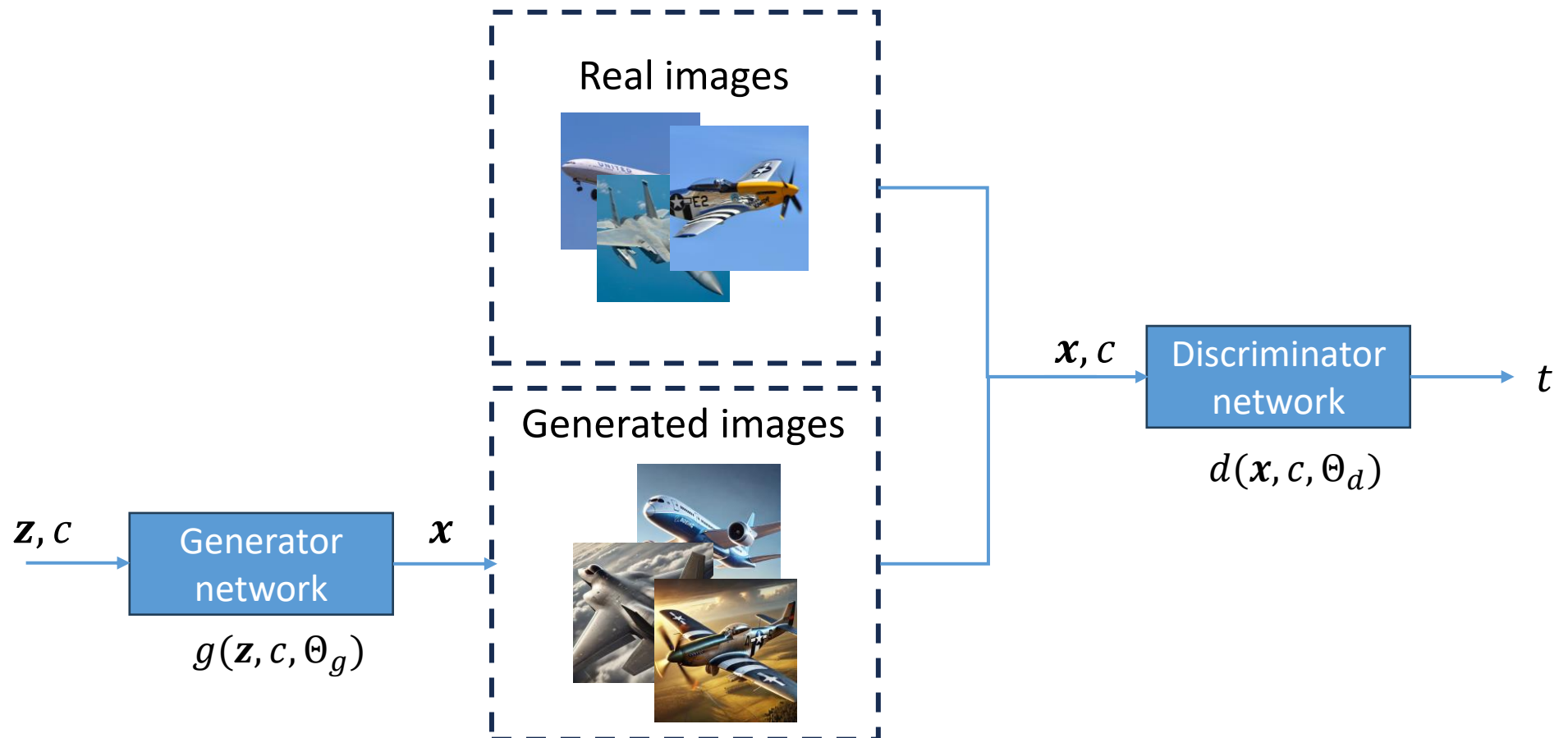
The updates for the generator are:

$$\Delta \Theta_g = \alpha \nabla_{\Theta_g} E$$

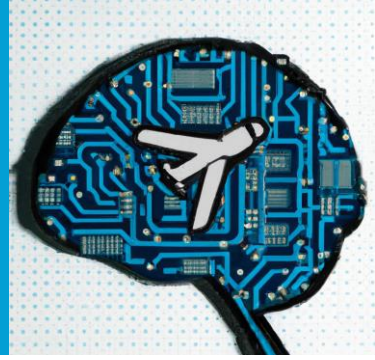
Conditional generation



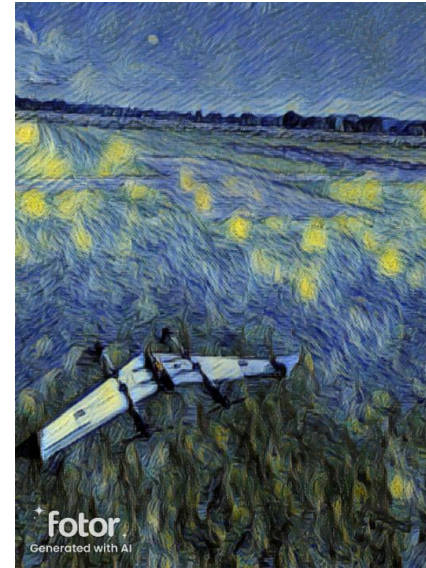
Add the class conditional to the inputs of both networks:



CycleGAN

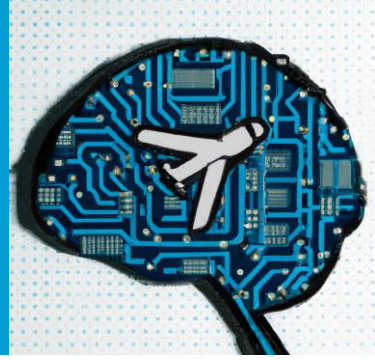


How to transform photos to paintings in a specific style?

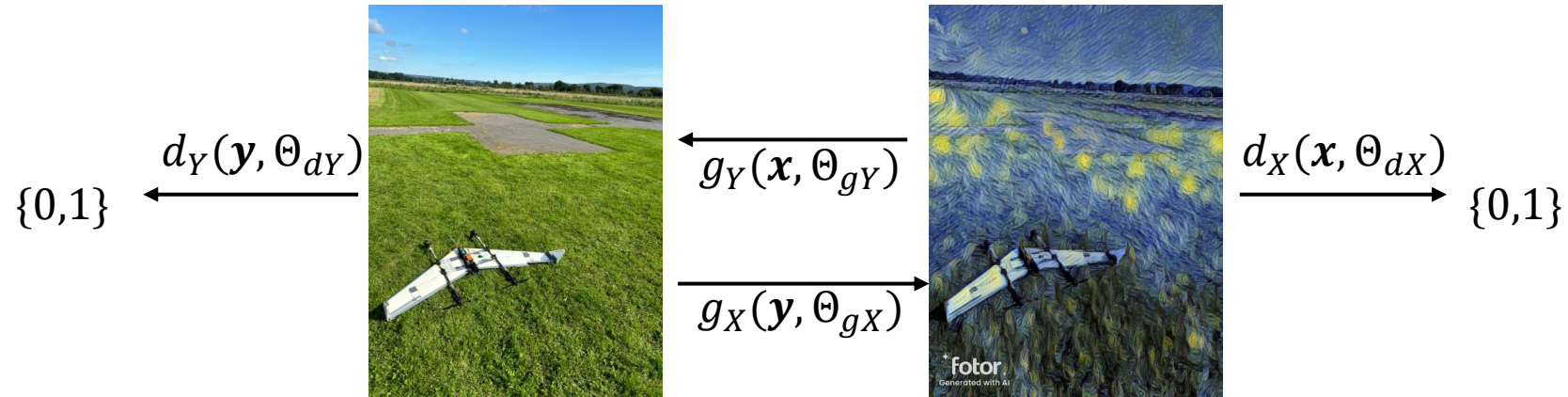


Generated with Fotor

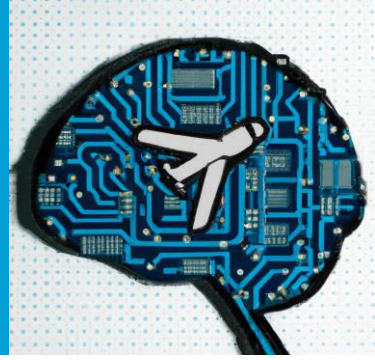
CycleGAN



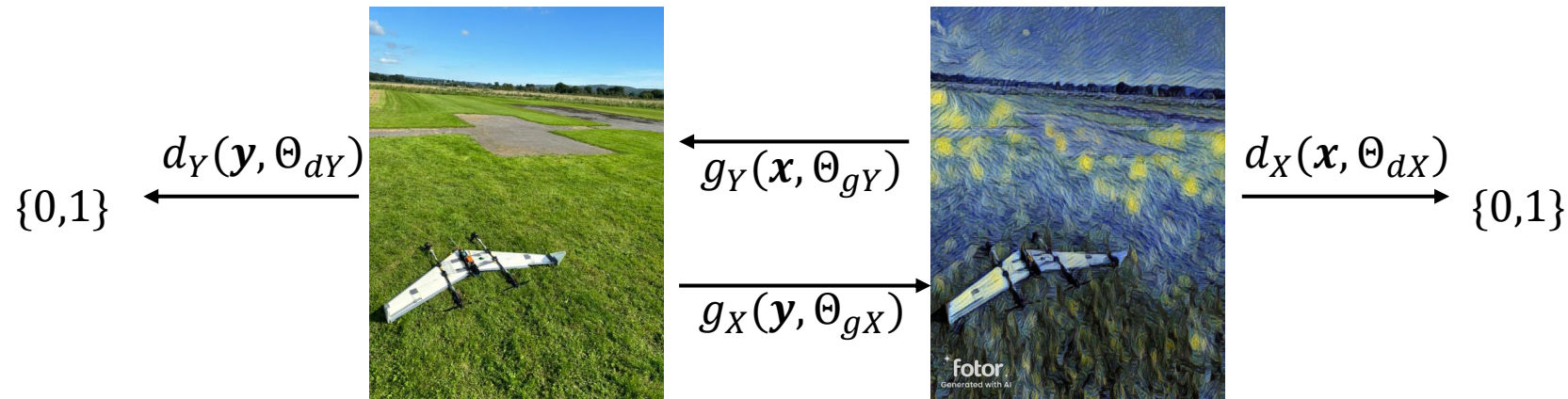
(1) Use two generators and two discriminators.



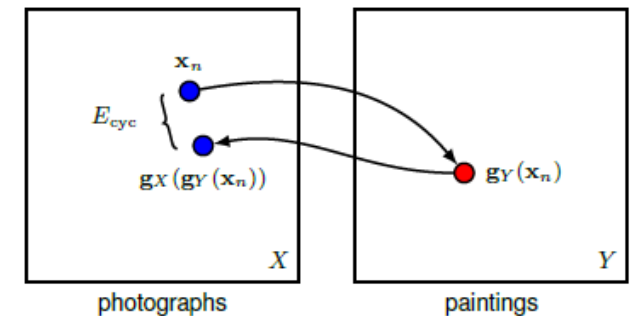
CycleGAN



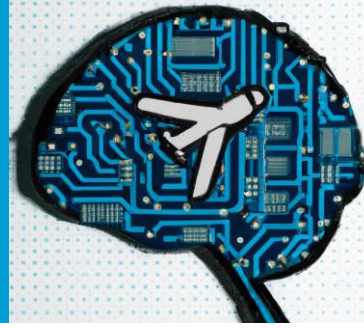
(1) Use two generators and two discriminators.



(2) Ensure cycle consistency, i.e., that the mapping $g_X(g_Y(\mathbf{x})) \approx \mathbf{x}$ and, similarly, $g_Y(g_X(\mathbf{y})) \approx \mathbf{y}$.



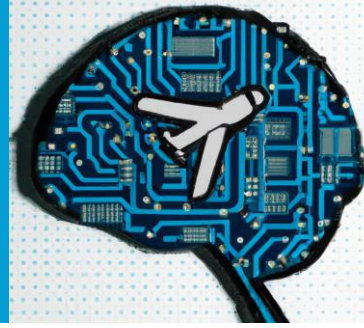
CycleGAN



The cyclic loss function is:

$$\mathcal{L}_{\text{cyc}} = \frac{1}{N_X} \sum_{i \in X} \|g_X(g_Y(\mathbf{x}_i)) - \mathbf{x}_i\|_1 + \frac{1}{N_Y} \sum_{i \in Y} \|g_Y(g_X(\mathbf{y}_i)) - \mathbf{y}_i\|_1$$

CycleGAN



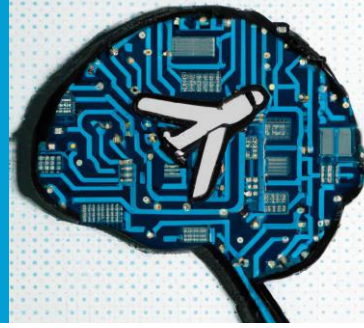
The cyclic loss function is:

$$\mathcal{L}_{\text{cyc}} = \frac{1}{N_X} \sum_{i \in X} \|g_X(g_Y(\mathbf{x}_i)) - \mathbf{x}_i\|_1 + \frac{1}{N_Y} \sum_{i \in Y} \|g_Y(g_X(\mathbf{y}_i)) - \mathbf{y}_i\|_1$$

Which is added to the normal GAN loss $\mathcal{L}_X$ and $\mathcal{L}_Y$ with a gain β

$$\mathcal{L} = \mathcal{L}_X + \beta \mathcal{L}_{\text{cyc}}$$

GANs



Advantages:



...



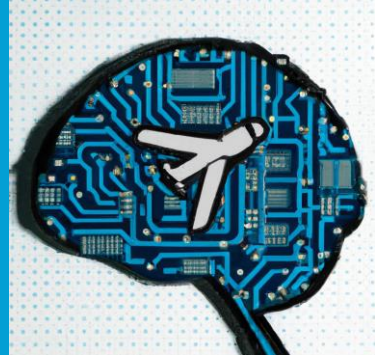
...

Disadvantages:



...

GANs



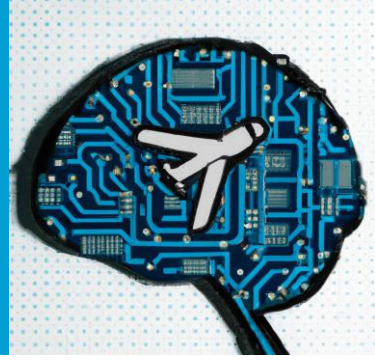
Advantages:

- ✓ Conceptually clear
- ✓ Can lead to high-quality results

Disadvantages:

✗ ...

GANs



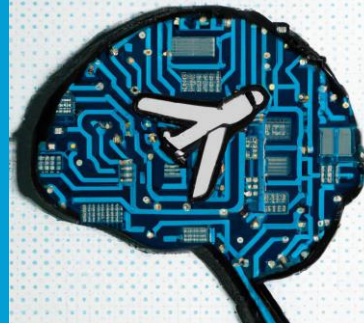
Advantages:

- ✓ Conceptually clear
- ✓ Can lead to high-quality results

Disadvantages:

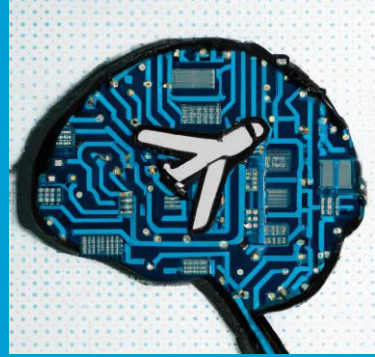
- ✗ Training can be very difficult

GAN training problems



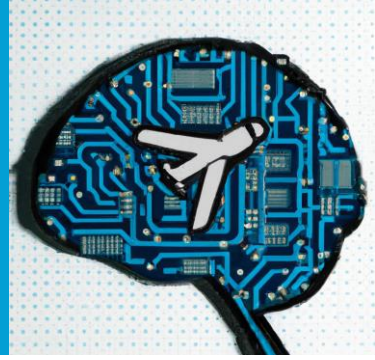
In general it makes for quite an unstable learning problem:

- No clear progress, as the error can go both up and down
- Training can get stuck in a region where either one of the networks is consistently winning and learning is slow
- Mode collapse: generator can start focusing on the few samples at which it is good (e.g., always the same “3” or “5” on MNIST)



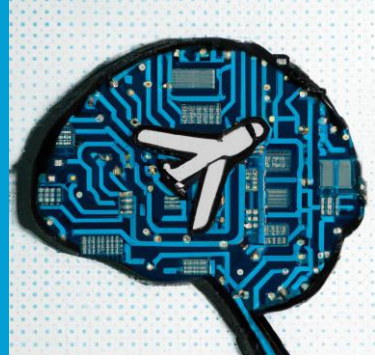
Diffusion models

Central idea of diffusion models



Generate a sample in **steps** instead of in one “go”.

Central idea of diffusion models



Generate a sample in **steps** instead of in one “go”.

A multi-step noise process takes a sample and transforms it step by step in a white-noise sample.

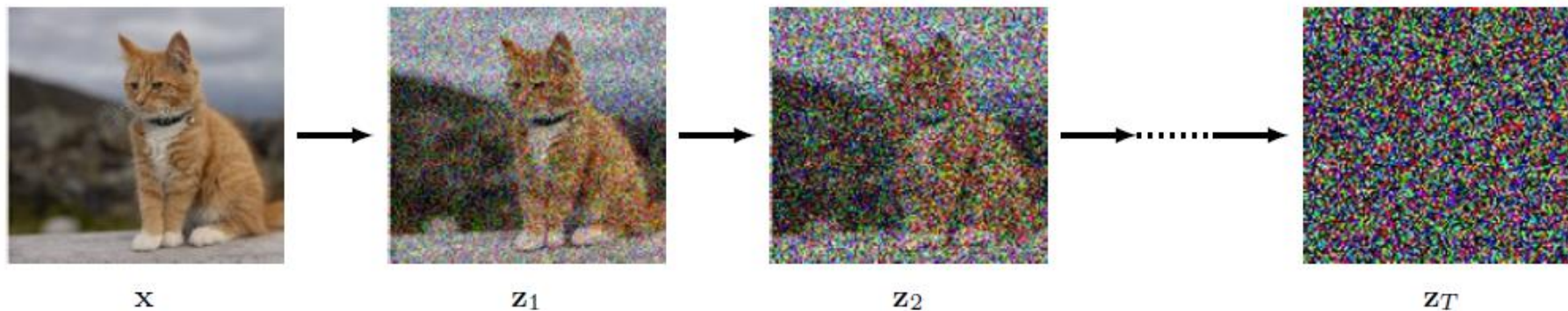
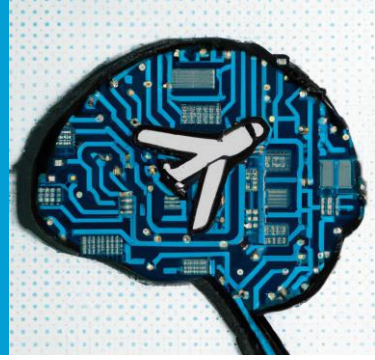


Figure from Bishop, C. M., & Bishop, H. (2023). Deep learning: Foundations and concepts. Springer Nature.

Central idea of diffusion models



Generate a sample in **steps** instead of in one “go”.

A multi-step noise process takes a sample and transforms it step by step in a white-noise sample.

A neural network then learns how to **revert** such steps.

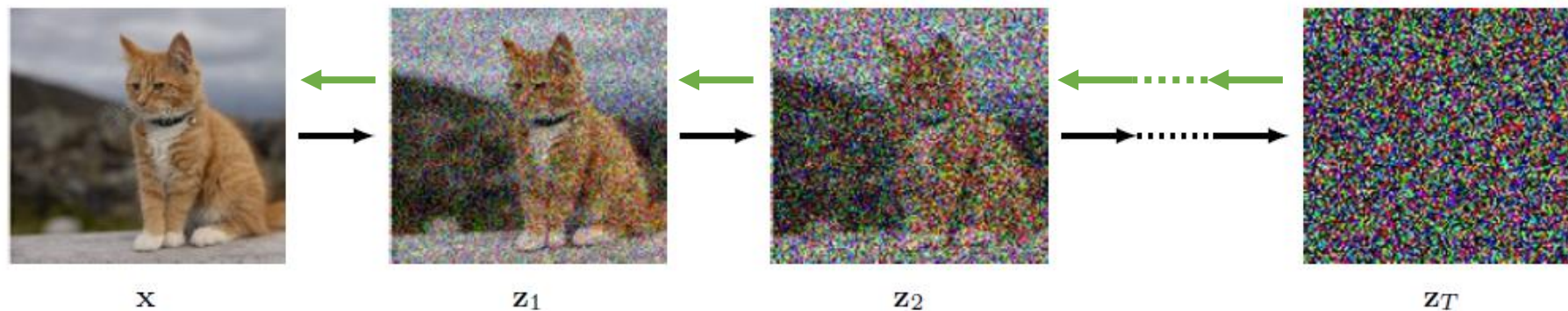
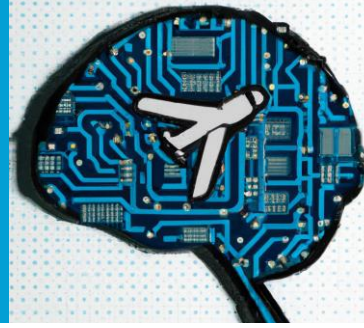
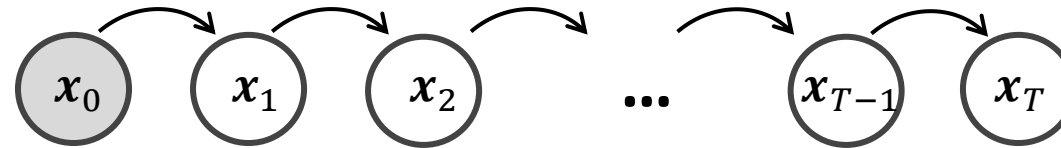


Figure from Bishop, C. M., & Bishop, H. (2023). Deep learning: Foundations and concepts. Springer Nature.

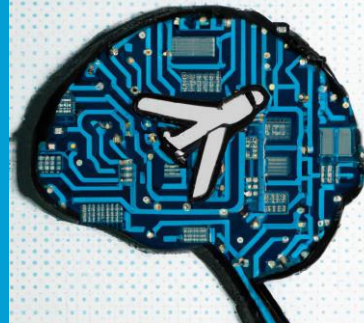
Diffusion process as a Markov chain



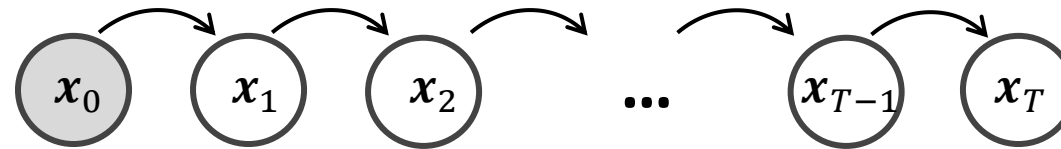
The forward diffusion process is fixed, determined by hyperparameters β .



Diffusion process as a Markov chain



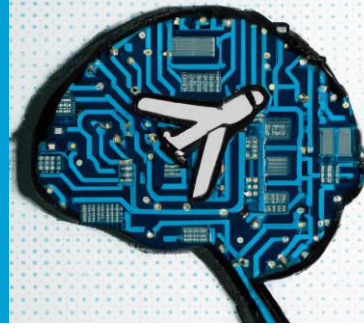
The forward diffusion process is fixed, determined by hyperparameters β .



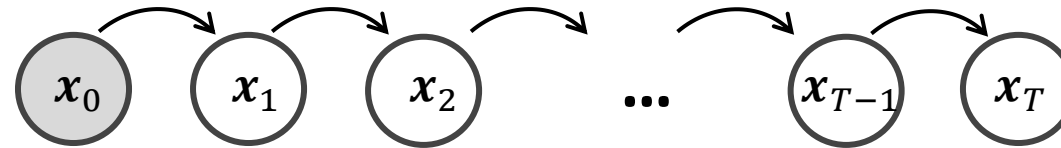
The forward transition is modeled by:

$$p(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N} \left(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I \right)$$

Diffusion process as a Markov chain



The forward diffusion process is fixed, determined by hyperparameters β .



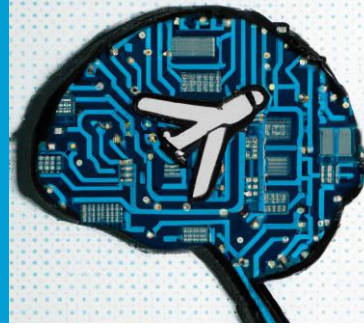
The forward transition is modeled by:

$$p(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I)$$

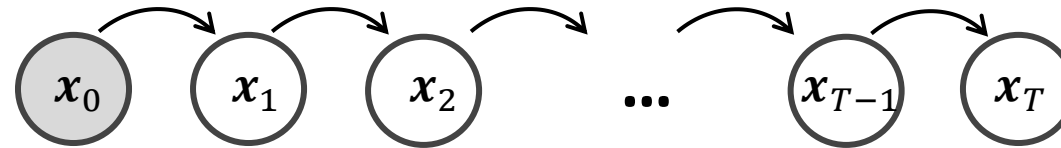
$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \beta_t \epsilon$$

with $\epsilon \sim \mathcal{N}(\mathbf{0}, I)$.

Diffusion process as a Markov chain



The forward diffusion process is fixed, determined by hyperparameters β .



The forward transition is modeled by:

$$p(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I)$$

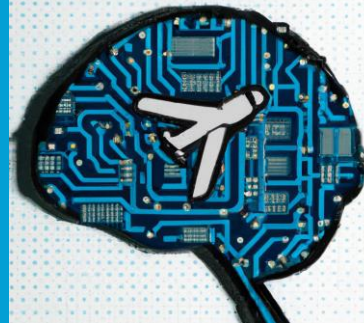
$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \beta_t \epsilon$$

with $\epsilon \sim \mathcal{N}(\mathbf{0}, I)$.

Note:

Multiplying by $\sqrt{1 - \beta_t}$ implies that after many steps, $\mathbf{x}_T$ will converge to $\mathbf{0}$.

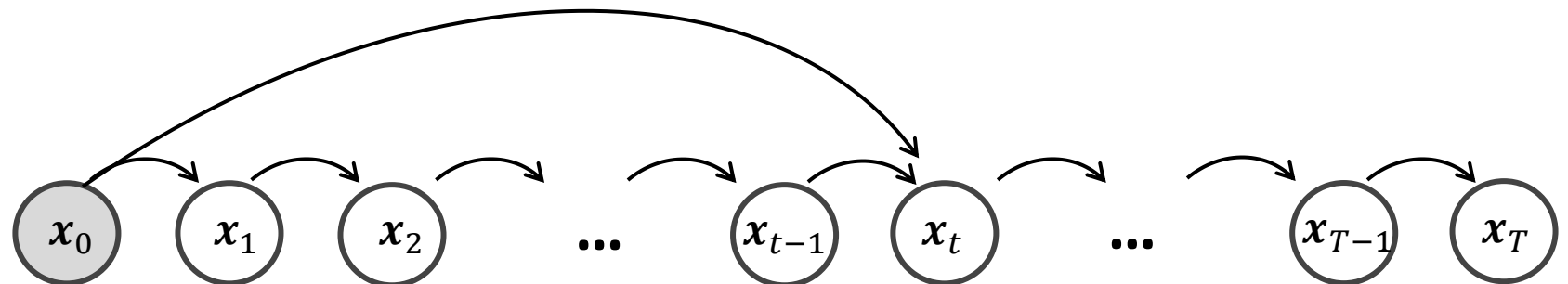
Diffusion process as a Markov chain



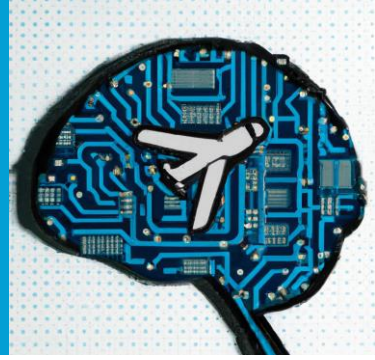
Given the fixed forward diffusion process, we can analytically resolve the probability distribution for a state after t steps:

$$p(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\alpha_t} \mathbf{x}_0, (1 - \alpha_t) I)$$

Where $\alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$.

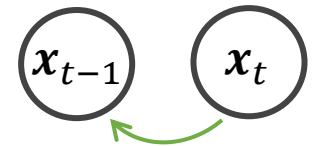


Reverse diffusion process

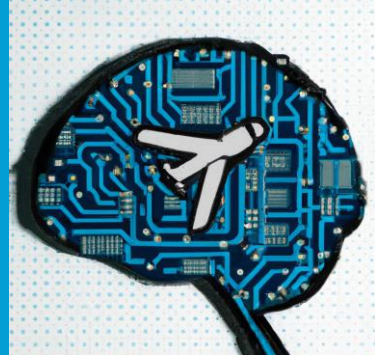


If the variances β_t are sufficiently small, the reverse distribution is Gaussian as well, with approximately the same covariance matrix:

$$P(\mathbf{x}_{t-1}|\mathbf{x}_t) \sim \mathcal{N}(\boldsymbol{\mu}, \beta_t I)$$

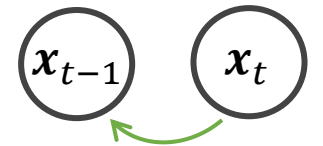


Reverse diffusion process



If the variances β_t are sufficiently small, the reverse distribution is Gaussian as well, with approximately the same covariance matrix:

$$P(\mathbf{x}_{t-1}|\mathbf{x}_t) \sim \mathcal{N}(\boldsymbol{\mu}, \beta_t I)$$

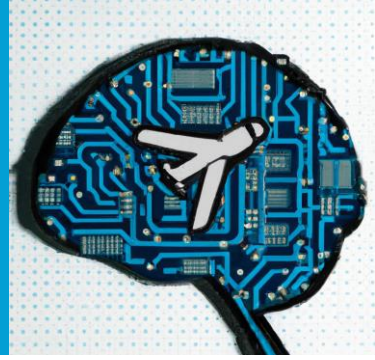


The idea now is to predict $\boldsymbol{\mu}$ with a deep neural network:

$$\boldsymbol{\mu}(\mathbf{x}_t, \Theta, t)$$

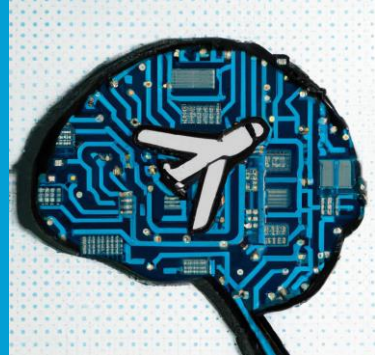
Where Θ are the network parameters, and t is necessary due the time step dependence of β_t .

How to train the network?



Ideally, one would like to match $P(\mathbf{x}_{t-1}|\mathbf{x}_t)$. However, this distribution cannot be determined in a tractable manner.

How to train the network?



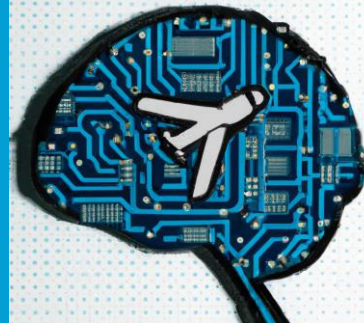
Ideally, one would like to match $P(\mathbf{x}_{t-1}|\mathbf{x}_t)$. However, this distribution cannot be determined in a tractable manner.

Instead, $P(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is used, since during training $\mathbf{x}_0$ is available.

$$P(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{m}_t(\mathbf{x}_0, \mathbf{x}_t), \sigma_t^2 I)$$

$$\text{With: } \sigma_t^2 = \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_{t-1}}.$$

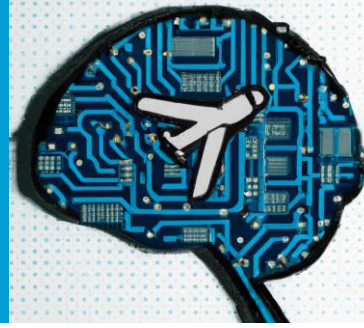
Loss function for the network



Furthermore:

$$\mathbf{m}_t(\mathbf{x}_0, \mathbf{x}_t) = \frac{(1 - \alpha_{t-1})\sqrt{1 - \beta_t}\mathbf{x}_t + \sqrt{\alpha_{t-1}}\beta_t\mathbf{x}_0}{1 - \alpha_t}$$

Loss function for the network



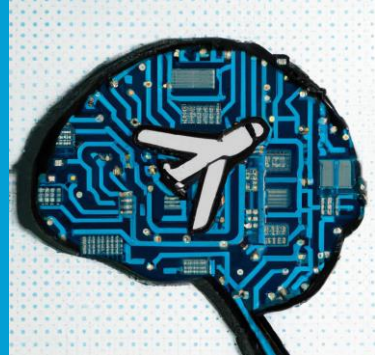
Furthermore:

$$\mathbf{m}_t(\mathbf{x}_0, \mathbf{x}_t) = \frac{(1 - \alpha_{t-1})\sqrt{1 - \beta_t}\mathbf{x}_t + \sqrt{\alpha_{t-1}}\beta_t\mathbf{x}_0}{1 - \alpha_t}$$

We minimize the **Kullback-Leibler divergence** between this conditional distribution and the network-based distribution:

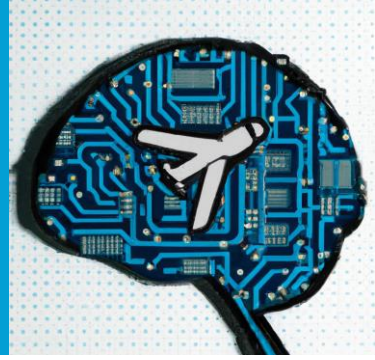
$$\text{KL}(P(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel P(\mathbf{x}_{t-1}|\mathbf{x}_t, \Theta)) = \frac{1}{2\beta_t} \|\mathbf{m}_t - \boldsymbol{\mu}(\mathbf{x}_t, \Theta, t)\|^2 + \text{const.}$$

Alternative training method



Idea: instead of estimating the next denoised image, predict the total noise ϵ that was added to x_0 .

Alternative training method

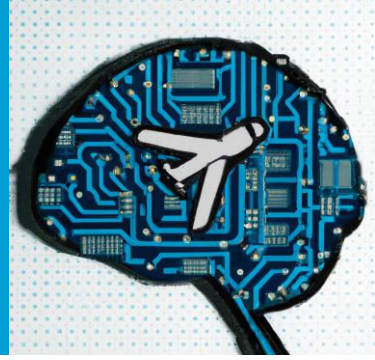


Idea: instead of estimating the next denoised image, predict the total noise ϵ that was added to $\mathbf{x}_0$.

Based on $p(\mathbf{x}_t | \mathbf{x}_0)$:

$$\mathbf{x}_0 = \frac{1}{\sqrt{\alpha_t}} \mathbf{x}_t - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \boldsymbol{\epsilon}_t$$

Alternative training method



Idea: instead of estimating the next denoised image, predict the total noise ϵ that was added to $\mathbf{x}_0$.

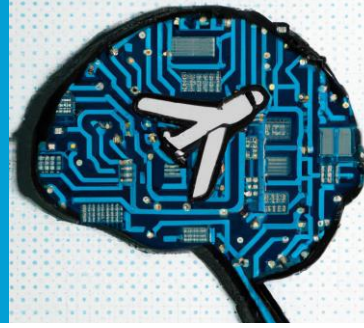
Based on $p(\mathbf{x}_t | \mathbf{x}_0)$:

$$\mathbf{x}_0 = \frac{1}{\sqrt{\alpha_t}} \mathbf{x}_t - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \boldsymbol{\epsilon}_t$$

Consequently:

$$\mathbf{m}_t(\mathbf{x}_0, \mathbf{x}_t) = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \boldsymbol{\epsilon}_t \right)$$

Alternative training method



Idea: instead of estimating the next denoised image, predict the total noise ϵ that was added to $\mathbf{x}_0$.

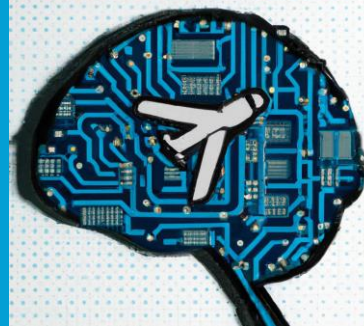
Based on $p(\mathbf{x}_t | \mathbf{x}_0)$:

$$\mathbf{x}_0 = \frac{1}{\sqrt{\alpha_t}} \mathbf{x}_t - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \boldsymbol{\epsilon}_t$$

Consequently:

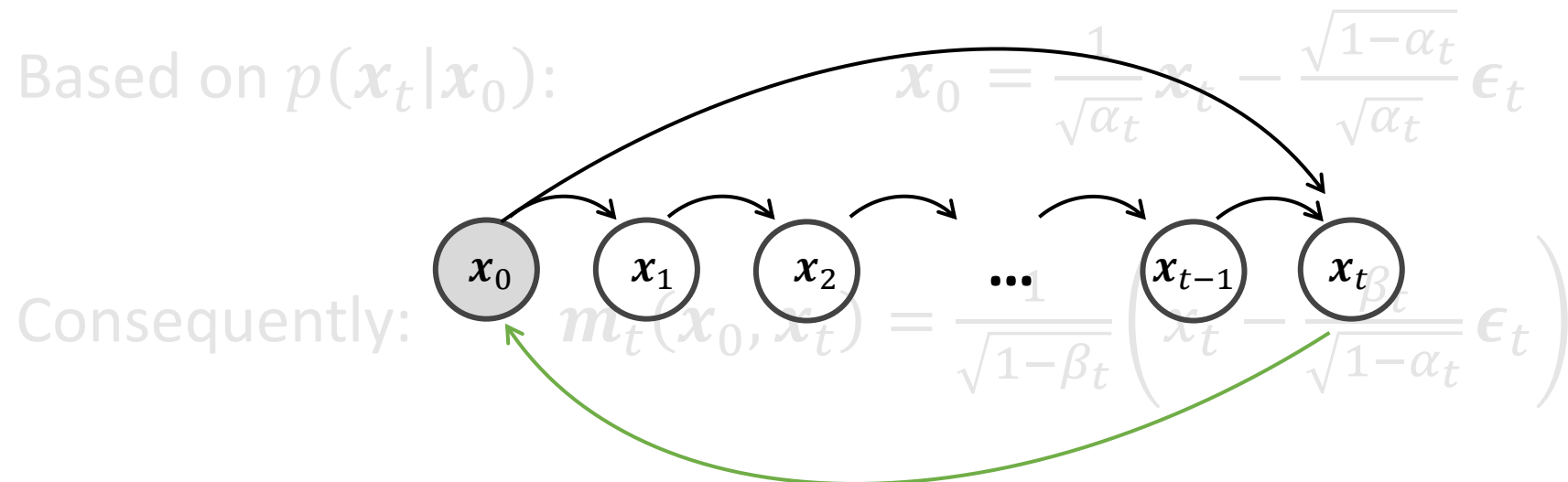
$$\mathbf{m}_t(\mathbf{x}_0, \mathbf{x}_t) = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \boldsymbol{\epsilon}_t \right)$$

Where now the network will estimate $\hat{\boldsymbol{\epsilon}}_t = f(\mathbf{x}_t, \Theta, t)$.



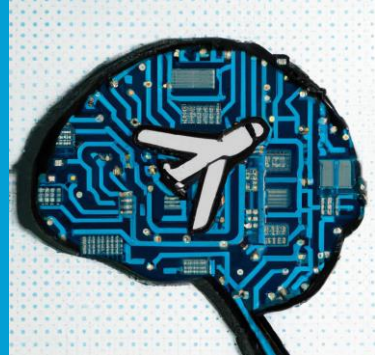
Alternative training method

Idea: instead of estimating the next denoised image, predict the total noise ϵ that was added to x_0 .



Where now the network will estimate $\hat{\epsilon}_t = f(x_t, \Theta, t)$.

Loss function

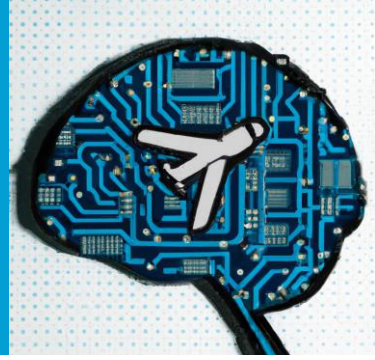


Minimizing the Kullback-Leibler divergence leads to the loss function:

$$\mathcal{L} = \sum_{i=1}^N \|f(\mathbf{x}_t, \Theta, t) - \epsilon_t\|^2$$

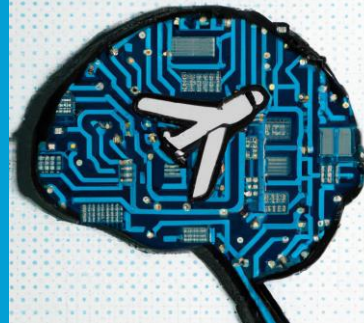
Where N samples are taken with randomly selected $t \in \{1, 2, \dots, T\}$.

Generating a sample – after training



Start with $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \beta_T I)$

Generating a sample – after training

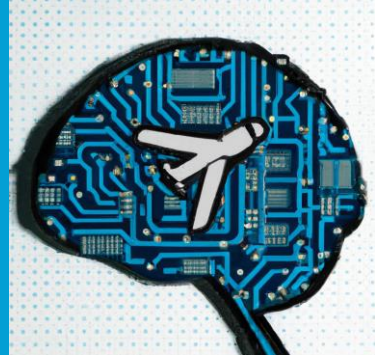


Start with $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \beta_T I)$

Repeat:

$$\hat{\epsilon}_t = f(\mathbf{x}_t, \Theta, t)$$

Generating a sample – after training



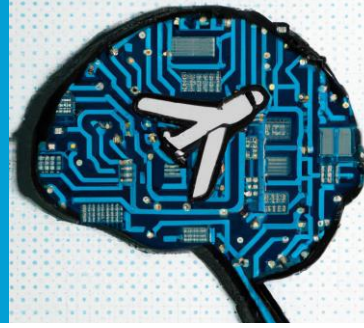
Start with $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \beta_T I)$

Repeat:

$$\hat{\boldsymbol{\epsilon}}_t = f(\mathbf{x}_t, \Theta, t)$$

$$\boldsymbol{\mu}_t = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \hat{\boldsymbol{\epsilon}}_t \right)$$

Generating a sample – after training



Start with $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \beta_T I)$

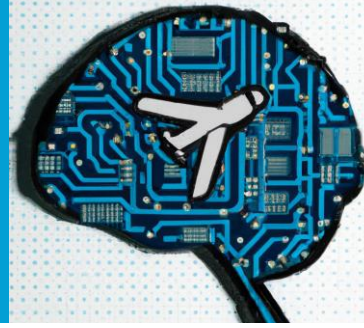
Repeat:

$$\hat{\boldsymbol{\epsilon}}_t = f(\mathbf{x}_t, \Theta, t)$$

$$\boldsymbol{\mu}_t = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \hat{\boldsymbol{\epsilon}}_t \right)$$

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_t + \sqrt{\beta_t} \boldsymbol{\epsilon}, \text{ with } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$$

Generating a sample – after training



Start with $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \beta_T I)$

Repeat:

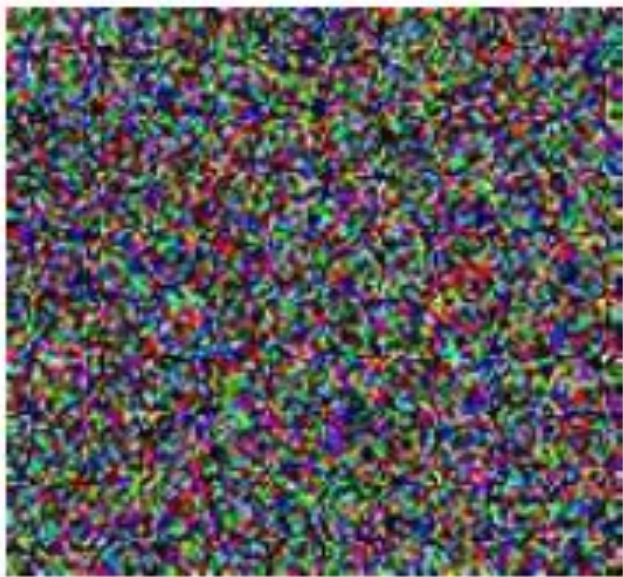
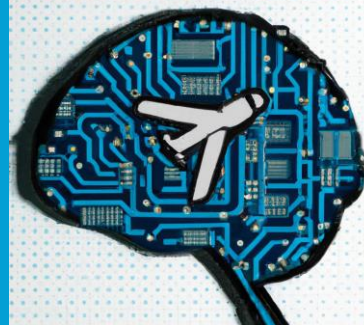
$$\hat{\boldsymbol{\epsilon}}_t = f(\mathbf{x}_t, \Theta, t)$$

$$\boldsymbol{\mu}_t = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \hat{\boldsymbol{\epsilon}}_t \right)$$

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_t + \sqrt{\beta_t} \boldsymbol{\epsilon}, \text{ with } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$$

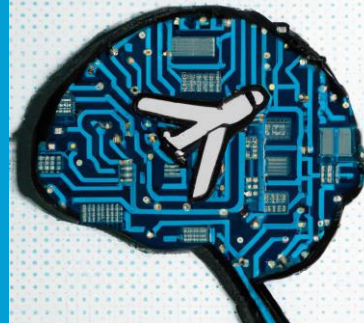
The last step, use a deterministic update: $\mathbf{x}_0 = \boldsymbol{\mu}_1$

Cat or Dog?



z_T

Conditional sample generation



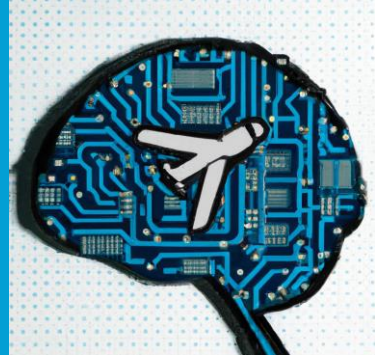
Classifier guidance

Train an unconditional diffusion model and add at each time step the scaled gradient of a classifier to the newly proposed input at the previous time step:

$$\hat{\epsilon} \leftarrow \epsilon(x_t, \Theta, t) - s \nabla_x \log P(y|x_t)$$

Where y can be, e.g., a class, and $\log P(y|x_t)$ the output of a classifier network.

Conditional sample generation

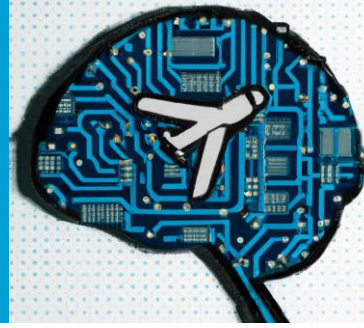


Classifier-free guidance

Provide class information as input to the neural network during training:

$$\hat{\epsilon}_t = f(\mathbf{x}_t, \mathbf{c}, \Theta, t)$$

Conditional sample generation



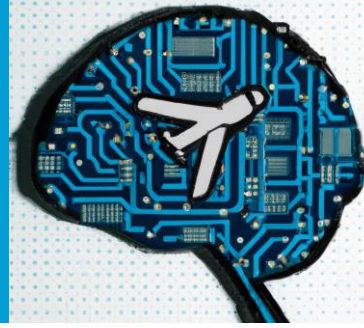
Classifier-free guidance

Provide class information as input to the neural network during training:

$$\hat{\epsilon}_t = f(\mathbf{x}_t, \mathbf{c}, \Theta, t)$$

With a certain probability, set the class vector to the null vector: $\mathbf{c} = \mathbf{0}$

Conditional sample generation



Classifier-free guidance

Provide class information as input to the neural network during training:

$$\hat{\epsilon}_t = f(\mathbf{x}_t, \mathbf{c}, \Theta, t)$$

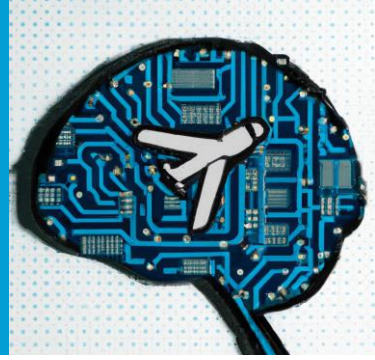
During training: with a certain probability, set the class vector to the null vector: $\mathbf{c} = \mathbf{0}$ to learn conditional (Class only) and unconditional noise (over all data)

When generating samples, use:

$$\hat{\epsilon} \leftarrow \gamma \epsilon(\mathbf{x}_t, \mathbf{c}, \Theta, t) + (1 - \gamma) \epsilon(\mathbf{x}_t, \mathbf{0}, \Theta, t)$$

, with $\gamma > 1$.

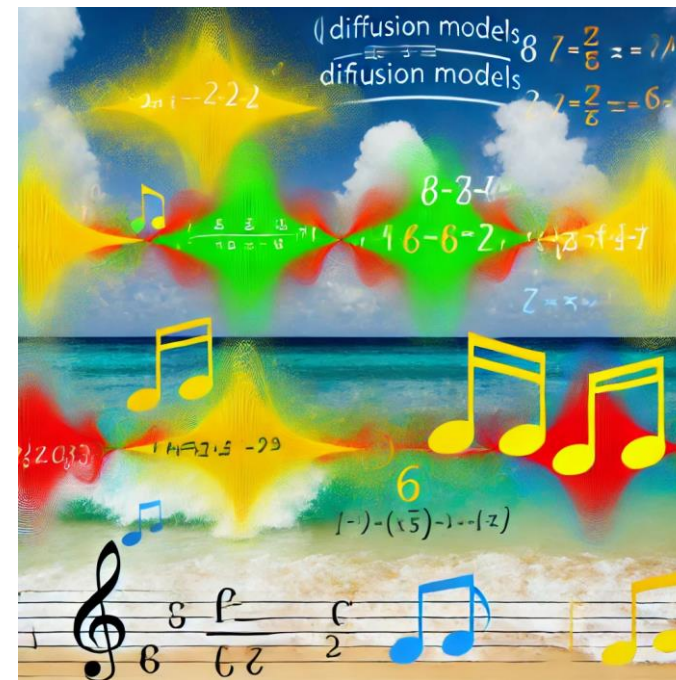
Results



All around us now – from generated text and images to music.



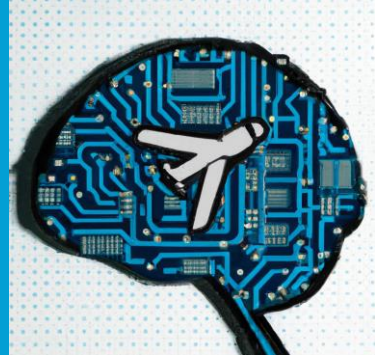
Stable Cascade (one of many image generation tools now)



Suno: “a funny reggae song about how fun it is to learn complex math and computer science to program diffusion models”



Diffusion models



Advantages:



...



...



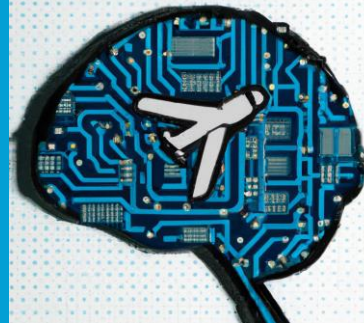
...

Disadvantages:



...

Diffusion models



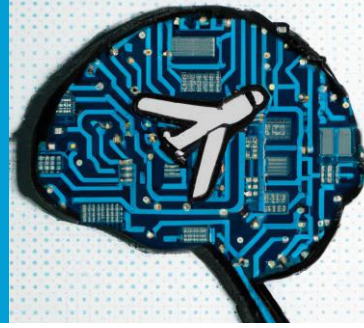
Advantages:

- ✓ Easy to train
- ✓ Stable training (in comparison to GANs)
- ✓ High-quality samples (comparable to GANs)

Disadvantages:

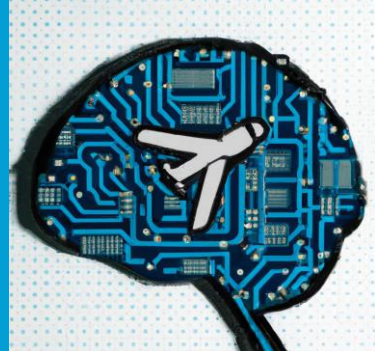
- ✗ The multi-step process leads to a large computational effort

Conclusion



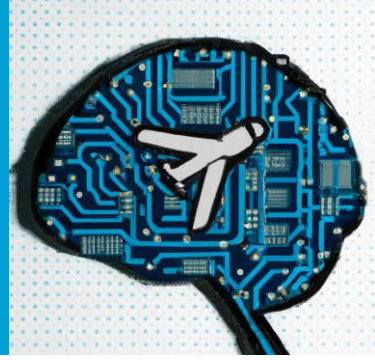
Generative models are very promising for AI, since they can capture relations in input data that go beyond the learned task.

The current state-of-the-art is impressive, but there are still challenges trading off generation quality (including preventing “hallucinations”), robust learning, and computational efficiency.



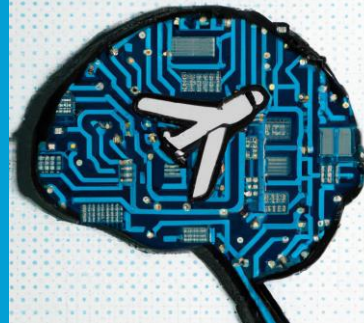
Quiz

Question



When you ask a generative model to generate an image that is similar to the previous one, you may actually get quite a different image. Why is this?

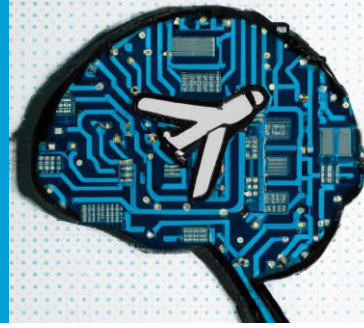
Question



When you ask a generative model to generate an image that is similar to the previous one, you may actually get quite a different image. Why is this?

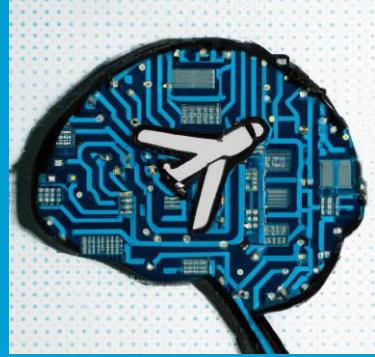
- A. The model might be using a different dataset each time it generates an image, leading to variations in the results.
- B. Generative models can only produce completely different images and cannot generate similar images by design.
- C. Generative models incorporate randomness in the generation process, which can lead to variations in the output even when using similar prompts.
- D. Minor differences in the input prompts or instructions can cause the model to produce significantly different outputs due to sensitivity in its interpretation of the prompts.

Question



When you ask a generative model to generate an image that is similar to the previous one, you may actually get quite a different image. Why is this?

- A. The model might be using a different dataset each time it generates an image, leading to variations in the results.
- B. Generative models can only produce completely different images and cannot generate similar images by design.
- C. **Generative models incorporate randomness in the generation process, which can lead to variations in the output even when using similar prompts.**
- D. Minor differences in the input prompts or instructions can cause the model to produce significantly different outputs due to sensitivity in its interpretation of the prompts.



Questions?